



StatsPAI: Validation-Tiered Python Workflows for Causal Inference

Biaoyue Wang 
Stanford University
and StatsPAI Inc.

Scott Rozelle 
Stanford University
and StatsPAI Inc.

Abstract

StatsPAI is a Python package for applied causal inference and econometrics in which every registered function carries an evidence tier that a user or an agent can query at call time. This article asks what such a label certifies and describes a design that makes the answer checkable. A label reaches the user through four mechanically verified links: the implementation a parity row exercised, established by a call trace bound to the source files on that path; the entry point the evidence is credited to; the configuration and output it compared, so a user can ask whether the variant and inference option just fitted were tested, and whether the standard error was, not only the point estimate; and the applicability of each automated check to the fit. Stochastic estimators are graded by seed-replicated equivalence against an explicit margin. The evidence covers a twelve-estimator core: same-byte parity with canonical R implementations (89 modules) and **Stata** bridges (85), known-truth recovery, Monte Carlo coverage reported with bias and standard-error calibration, and timings on shared inputs that keep unfavourable comparisons. Applying the design to the package found defects that numerical agreement alone had not, including parity rows that compared R with itself, an IV entry point that silently ignored a requested cluster-robust covariance, and a fixed-effects default whose clustered standard errors over-covered. Seven worked examples, executed verbatim, and a licence-free replication path complete the article. The evidence for the agent-facing interface is mechanical and contractual, not a claim of behavioural agent superiority.

Keywords: causal inference, applied econometrics, validation tiers, Python, R, Stata, software, machine learning, agent-facing registry API, large language models, reproducibility.

1. Introduction

Applied causal work now spans a large and fragmented software stack: **Stata** commands, methodological-reference R packages, and an expanding but incomplete Python ecosystem.

A single project can require OLS and IV, staggered difference-in-differences, regression discontinuity, synthetic control, matching, double machine learning, sensitivity analysis, and publication tables. The statistical literature has converged toward more explicit design checks and estimator-specific inference (Imbens and Wooldridge 2009; Angrist and Pischke 2009; Callaway and Sant’Anna 2021; Roth, Sant’Anna, Bilinski, and Poe 2023; Wing, Yozwiak, Hollingsworth, Freedman, and Simon 2024; Baker, Callaway, Cunningham, Goodman-Bacon, and Sant’Anna 2026; Rambachan and Roth 2023; Abadie, Diamond, and Hainmueller 2010; Chernozhukov, Chetverikov, Demirer, Duflo, Hansen, Newey, and Robins 2018); the software interface has not converged with it. Users still cross package boundaries, result classes, naming conventions, and diagnostic vocabularies. Large-language-model agents add a further constraint: they need typed schemas and structured errors, not prose-only help pages.

This paper introduces **StatsPAI**, a Python platform for applied econometrics and causal inference. The package provides (i) a unified `import statspai as sp` surface across common causal designs, (ii) shared reporting conventions for mature estimator results, diagnostics, citations, and export methods, (iii) a machine-readable registry that can be exposed through a Model Context Protocol (MCP) server (Anthropic 2024), and (iv) an explicit validation discipline against analytical targets, canonical R implementations, and selected Stata migration bridges. This article describes **StatsPAI** 1.32.0, distributed on PyPI (`pip install statspai==1.32.0`) under the MIT licence; the source is tag v1.32.0 of <https://github.com/brycewang-stanford/StatsPAI>, and every **StatsPAI** number in the article was produced by that release, against the pinned reference environments of Section 8.

A short paper on the package appeared in the *Journal of Open Source Software* (Wang and Rozelle 2026). It established the unified interface, the registry and agent schemas, the structured result objects, and the existence of a same-byte cross-language parity harness and of a 1,000-replication coverage run for OLS, DiD, and IV. This article does not re-establish those facts. Its question is the one a parity harness raises but cannot answer by itself: *what exactly does a validation label certify?* A green parity row can be earned by the wrong code (a Python wrapper that calls the R reference it is compared against), vouch for the wrong function (an alias credited with evidence that ran another entry point), rest on the wrong statistical comparison (a stochastic estimator judged against its sampling standard error rather than its own Monte Carlo error), or describe a configuration or an output the user is not relying on (a parity row for a coefficient cited as evidence for its standard error). The contribution is a design that makes these failure modes checkable, applied to the package’s own validated core. A label reaches a user through four links, and each is verified rather than asserted (Figure 3): the *implementation* a parity row exercised, established by a call trace bound to the source files on that path (Section 5.3); the *entry point*, since an inherited label must name a test that ran the inheriting function (Section 5.8); the *configuration and output*, since `sp.validation_scope` reports, for a fitted object of the twelve-estimator core, which artifacts ran its variant, inference option, and code path and which outputs—point estimate, standard error, interval coverage, diagnostic—they compared (Section 5.4); and *applicability*, since automated diagnostics report a check that does not exist for a fit as not applicable rather than missing (Section 4.1).

The software and the validation design are one contribution, not two. The design is not proposed in the abstract and tried on an example: it is implemented inside **StatsPAI**, its labels are returned by the same calls that fit the models, and each link above is held by a test in the package’s own suite. Conversely, the package’s usefulness for applied work rests on those

labels, because a broad interface is only as trustworthy as a user's ability to tell its validated core from its long tail at the moment of use. The design itself carries over to any package that publishes evidence tiers.

Around these links the article fixes an adjudication procedure for disagreements (Section 5.7), reserves the stochastic grade T3 for seed-replicated equivalence against a stated margin, and reports evidence the JOSS paper does not contain: estimand and identification contracts for the core, coverage diagnostics that explain a rate rather than grade it (bias, Monte Carlo SD, standard-error calibration, interval length), a timing study on shared inputs that keeps the unfavourable comparisons, and seven executed worked examples. Applying the design to its own package found defects that numerical parity alone had not: two Honest-DiD rows that compared R with itself; a coverage row whose entry point reported a separate AIPW estimator rather than the forest's; an entropy-balancing standard error twice the estimator's Monte Carlo SD; an audit that asked for an over-identification test on a just-identified model; an IV entry point that silently ignored a requested small-sample cluster covariance; and a fixed-effects default whose clustered standard errors no parity row had exercised and which over-covered (0.979 at nominal 0.95).

The registry distinguishes `certified`, `validated`, `api_stable`, and `experimental` symbols – queryable at call time – so numerical validation is an evidence tier rather than a blanket claim over the whole API surface; the full census opens Section 5.

Listing 1 fixes ideas. The classical Card (1995) returns-to-schooling design is estimated four ways — OLS, instrumental variables, double machine learning (Chernozhukov *et al.* 2018), and a causal forest (Wager and Athey 2018) — from a single import, and three of the four fitted objects, which belong to three different result classes, are rendered into one publication table by a single call. Three design properties recur throughout the paper: the estimator surface is *flat* (classical and modern methods are siblings under `sp.`); the result objects are *interoperable* (mixed estimators feed directly into one `sp.regtable` call); and the platform exposes *audit hooks* (`sp.audit`) that list the diagnostic evidence a design demands but the user has not yet supplied. Section 4.1 runs the same session at a prompt and prints what each of these calls returns, and Section 4.3 measures what the shared result contract buys on a harder case: rebuilding an event-study covariance from reported standard errors, the natural shortcut when a sensitivity analysis is composed by hand, overstates a Honest-DiD breakdown value by 26%.

Every listing in this paper is executed verbatim by the replication archive's listing audit, and for the transcripts the audit goes further: it replays each one statement by statement through a real interpreter loop and compares the output printed here, line for line, with the output the code actually produces (Section 8.2). A printed transcript is quoted as evidence and read as evidence, so it is checked like evidence.

```
import statspai as sp

card = sp.datasets.card_1995()
X = ["exper", "expersq", "black", "south", "smsa"]

ols = sp.regress("lwage ~ educ + " + " + ".join(X), data=card)
iv = sp.iv("lwage ~ " + " + ".join(X) + " + (educ ~ nearc4)",
           data=card)
dml = sp.dml(card, y="lwage", d="educ", X=X,
```

```

        model="plr", n_folds=5, random_state=42)
cf = sp.causal_forest("lwage ~ educ | " + " + ".join(X), data=card,
                     n_estimators=2000, discrete_treatment=False,
                     random_state=42)

table = sp.regtable(ols, iv, dml, keep=["educ", "theta"], stars=True,
                   model_labels=["OLS", "2SLS", "DML PLR"])
table.to_docx("returns_to_schooling.docx")

```

Listing 1: **StatsPAI** reproduces and extends Card (1995) from a single import; classical and modern estimators are siblings under **sp**. and return one interoperable result contract.

The package surface is deliberately broad, spanning classical regression and panel models, the modern DiD, RD, and synthetic-control literatures, ML-based treatment-effect estimation, matching and weighting, sensitivity analysis, and publication output. Breadth is not the paper’s evidentiary claim. In this paper, *validation evidence* means that a specific artifact exists: known-truth recovery, cross-language parity on identical data bytes, Monte Carlo coverage, or an explicit convention disclosure. Only the first two earn the **validated** and **certified** registry labels (Section 5); coverage rows and disclosures are recorded but earn nothing on their own. Unit and regression tests are API-contract evidence, and an **api_stable** helper has a stable call signature; neither acquires statistical credibility by proximity to validated estimators. The validated claim is limited to the twelve-estimator core of Section 5.1 and to the named rows of the parity ledger.

The closest implementations cover important parts of this terrain, but with different centers of gravity; Table 1 states the comparison directly. Empirical comparisons appear later in the same-byte parity rows (Tables 8 and 9) and in the timing study (Figure 4); per-entry-point detail is generated into `Paper-JSS/parity/replacement-table.md`. The contribution is therefore an integration-and-validation layer around reference software, not a claim that **StatsPAI** supersedes specialized packages or reference implementations in their home domains, such as **fixest**, **did**, **rdrobust**, **Synth**, **synthdid**, **DoubleML**, or **grf**. Representative software citations include **Synth**, **MatchIt**, **DoubleML**, **EconML**, **CausalML**, and **DoWhy** (Abadie, Diamond, and Hainmueller 2011; Ho, Imai, King, and Stuart 2011; Bach, Chernozhukov, Kurz, and Spindler 2022; Battocchi, Dillon, Hei, Lewis, Oka, Oprescu, and Syrgkanis 2019; Chen, Harinen, Lee, Yung, and Zhao 2020; Sharma and Kiciman 2020).

The replication archive carries the executable ledgers that decide whether a label is justified, together with a reviewer evidence map and editor screening checklist that route validation-scope, reproduction, and release-boundary questions to those artifacts.

Section 2 describes the architecture the evidence attaches to and Section 3 the agent-facing interface; Section 4 runs seven worked examples, with staggered DiD as the main case study; Sections 5–7 report the validation, performance, and interface evidence, and Section 8 the reproduction path.

2. Software architecture

StatsPAI is implemented in Python on top of **NumPy** (Harris *et al.* 2020), **SciPy** (Virtanen *et al.* 2020), **pandas** (McKinney 2010), **statsmodels** (Seabold and Perktold 2010), **scikit-learn** (Pedregosa *et al.* 2011), and **linearmodels**, with optional backends kept behind extras and

Comparable implementation	Where it remains the reference choice	StatsPAI contribution and boundary
statsmodels , linearmodels , pyfixest	Mature regression, panel, and HDFE APIs with fuller model-specific options.	A causal-workflow layer that standardizes cross-family handoffs, with audit prompts, citations, and agent schemas; not the fastest specialist path.
DoubleML , EconML , CausalML , grf	Deeper ML-causal and CATE surfaces and tuning controls.	DML and forests inside one result, audit, and evidence ledger; the forest is a T3 equivalence claim, not parity with every grf option.
differences , causalimpact , narrower workflow packages	Focused workflows for one model family.	One namespace and post-estimation contract across families, at the price of a larger dependency surface.
DoWhy	Graph-first identification and refutation.	Estimator- and evidence-led; no graph-identification automation.
did , fixest , rdrobust , Synth , synthdid , MatchIt ; csdid , reghdfe , sdid	Authoritative references in their home ecosystems, with more edge-case options.	Validated against them on identical bytes; records disagreements as ledger entries rather than strict parity claims (one T4 disclosure remains); Stata rows need a separate licensed runtime.

Table 1: Closest existing implementations and comparative scope, surveyed in May 2026 and spot-checked in September 2026.

lazy imports. The architecture has four layers: registry, dispatcher, result object, and output pipeline.

2.1. Design principles

A single flat namespace exposes routine calls through `sp.<function>`. Mature estimator paths return structured result objects rather than raw tuples, while auxiliary helpers (plotting and data inspection) remain registered with explicit stability and validation metadata. Method families use dispatchers (`sp.synth(method=...)`, `sp.dml(model=...)`). Reference parity is attempted before a validated claim is broadened; assumption violations raise typed exceptions or warnings and record diagnostics; output-changing changes go through the changelog and `MIGRATION.md`; and every public function is discoverable through the registry that serves both help functions and agent schemas.

2.2. Why an integration package?

A reasonable alternative is to compose **statsmodels**, **linearmodels**, **DoubleML**, **EconML**, **DoWhy**, and method-specific R or **Stata** bridges directly, which is often right for a single specialized model. The friction appears when an applied project moves across identification designs: diagnostics, citations, validation status, error handling, export formats, and agent schemas no longer share one contract. **StatsPAI** therefore treats wrapping as insufficient unless a method enters the common registry, result-object surface, typed-error envelope, and evidence ledger. Specialized packages remain the reference implementations; the contribution here is to make cross-family handoffs auditable and reproducible. The package is organized by identification layer rather than by the historical boundaries of R and **Stata** packages – each layer reports its own design diagnostics (balance and overlap, first-stage strength, cohort support and event-study paths, bandwidth and density checks, donor weights and placebos, cross-fit diagnostics) – and the common point is not shared identification assumptions but a

shared post-estimation contract.

2.3. Registry and discovery layer

The central data structure is a registry of `FunctionSpec` records. Each entry carries a canonical function path, category, tags, stability and validation status, parameter metadata, result class, reference key, example call, and optional agent-card fields such as assumptions, preconditions, failure modes, alternatives, limitations, and typical sample-size guidance. The registry powers `sp.help()`, `sp.list_functions()`, `sp.search_functions()`, `sp.describe_function()`, `sp.function_schema()`, `sp.agent_card()`, the import-free `schemas/` bundle, and the MCP tool catalogue. Tests assert that exported public callables are registered and that validated/certified entries expose their evidence status.

The validation tier is therefore not a paper-only artifact: it is queryable from the running software for *every* registered function. `sp.help()` prints a per-tier census and points the user to `sp.list_functions(validation_status="certified")` for the validated core, while `sp.describe_function(name)` prints a `Stability/Validation` header with the attached evidence notes before the description, so a user or agent landing on an `api_stable` or `experimental` helper sees its trust tier before relying on its output.

```
>>> spec = sp.describe_function("callaway_santanna")
>>> spec["stability"], spec["validation_status"]
('stable', 'certified')
>>> import textwrap
>>> notes = spec["validation_notes"]
>>> print(*notes[:2], sep="\n")
R parity module 04_csdid
Stata parity module 04_csdid
>>> print(textwrap.fill(notes[2], width=64))
Cross-language parity (bit-exact) vs did::att_gt + aggte [R
version 4.5.2 (2025-10-31), did 2.3.0]; tolerance
rel_est<=1e-06, rel_se<=1e-09; parity module 04_csdid --
tests/r_parity/04_csdid.py
>>> print(textwrap.fill(spec["limitations"][0], width=64))
clustervars is not yet supported with bstrap=False; the analytic
standard errors do not account for within-cluster dependence, so
the multiplier bootstrap is required
```

Listing 2: The validation tier is visible through the same discovery surface used by users and agents; this excerpt is generated from `sp.describe_function("callaway_santanna")`.

2.4. Family dispatchers

Family dispatchers reduce method sprawl without hiding assumptions. `sp.synth` dispatches classical, SDID, augmented, generalized, matrix-completion, penalized, and related synthetic-control variants. `sp.decompose` covers Blinder–Oaxaca, Gelbach, Fairlie, RIF, DFL, Machado–Mata, Melly, counterfactual-distribution, and related decompositions. `sp.dml` dispatches partially-linear, partially-linear IV, interactive, and LATE-style DML variants. The dispatcher contract is explicit: allowed `method=` values are enumerated in the registry schema, and validated or mature variants return compatible result surfaces so that summary, plotting,

audit, citation, and export calls do not change downstream.

The same uniformity extends to inference. Across the regression family (`regress`, `feols`, `ivreg`, `panel`, `hdfe_ols`), one canonical keyword, `vce=`, spells the full standard-error menu: classical, heteroskedasticity-robust, one- and two-way clustering, bias-reduced CR2/CR3 (Pustejovsky and Tipton 2018), the wild cluster bootstrap (Cameron, Gelbach, and Miller 2008; Roodman, Nielsen, MacKinnon, and Webb 2019), and Conley spatial HAC (Conley 1999); fixed-effects Poisson and GLM additionally expose CR2/CR3 and the restricted score wild bootstrap. Each option is pinned to an external reference implementation — Stata’s `reghdfe`, `boottest`, `acreg`, and `ppmlhdfe` (Correia, Guimarães, and Zylkin 2020), and R’s `clubSandwich` and `sandwich` (Zeileis, Köll, and Graham 2020) — and a coverage matrix in the package’s continuous integration ratchets the count of natively wired, externally validated cells monotonically upward.

2.5. Result-object hierarchy

Most estimators return either `EconometricResults` or `CausalResult`; specialized classes must implement the same public surface. The standard contract covers summaries, plotting, dictionary and JSON payloads, LaTeX/DOCX export, citations, audits, diagnostics, and degradation records. This is why a mixed OLS/IV/DML/forest table can be passed to `sp.regtable` (or its R-style alias `sp.modelsummary`), and why an MCP agent can keep only a result handle rather than copying coefficient and covariance arrays between tool calls.

2.6. Performance backbone

The performance-critical path is high-dimensional fixed-effect absorption for fixed-effect and panel estimators. The submitted artifact uses the reproducible Python path built on **NumPy** and **Numba** (Lam, Pitrou, and Seibert 2015). An experimental Rust crate and selected **JAX** and **PyTorch** paths exist behind extras, but the numerical and timing claims in this paper do not depend on a compiled native extension. Optional acceleration is treated as a backend choice, not as a separate estimator or a hidden requirement.

2.7. Output pipeline and workflow

Publication output is unified through `sp.regtable` and the result-level exporters, which render mixed result classes as LaTeX, Markdown, HTML, Word, Excel, or JSON-safe agent payloads. The layers combine into a four-stage workflow—detect (`sp.detect_design`, `sp.preflight`), fit (the dispatchers), inspect (`summary()`, `sp.audit()`, sensitivity wrappers), and report (`sp.regtable`, `cite()`, `sp.paper()`)—with the same boundaries for human users and MCP agents.

3. The agent-facing registry API

StatsPAI exposes the surface a human Python user calls to LLM agents through typed schemas and an MCP server. The agent layer is a software-interface contribution: it turns four otherwise opaque handoffs—tool discovery, failure recovery, result reuse, and citation resolution—into artifacts a reviewer can inspect, makes machine-orchestrated calls subject to the same validation ledger as human ones, and is validated by schema/MCP/citation

reproducibility rather than a model-quality score. Section 7 reports the mechanical evidence for it and what it does not show.

3.1. Schemas and agent cards

Docstrings, R help pages, and Stata help files are prose interfaces. A tool-using agent needs a typed object containing parameter names, types, defaults, required fields, enumerated choices, and a short description including stability and limitations when relevant. The schema call returns the executable OpenAI-style object; the description and agent-card calls return richer planning metadata such as assumptions, preconditions, failure modes, alternatives, typical sample size, references, and examples. These discovery functions are themselves registered, so an agent can inspect the estimator catalogue before selecting a method, and design-detection and preflight calls expose cheap planning diagnostics before an estimator is run.

3.2. Typed errors and the MCP server

Estimator failures inherit from `StatsPAIError` and carry structured fields: `error_kind`, `detail`, `recovery_hint`, `alternative_functions`, and any partial diagnostics. Warnings use the same idea for non-fatal degradations. The MCP server serializes these records as structured errors, so an agent can branch on `weak_instrument`, `no_overlap`, `rank_deficient`, or similar identifiers without scraping English messages.

The `statspai-mcp` server exposes registered functions as MCP tools. Tool inputs are generated from the registry schemas, outputs are JSON-safe result payloads plus `result_id` handles, and typed errors use the exception envelope above. Chained tools cover auditing, Honest-DiD sensitivity, generic sensitivity analysis, CS aggregation, Bacon decomposition, paper generation, and citation, and they accept result handles instead of requiring an agent to marshal coefficient, covariance, or cross-fit arrays through model context. The deterministic trace in Section 7 exercises this contract.

3.3. The estimand-first DSL and robustness audit

`sp.causal_question(...)` accepts a requested estimand, treatment, outcome, data, optional design, covariates, instruments, and sensitivity flag, then routes to the appropriate estimator and returns a `CausalQuestionResult`. The result bundles the fitted estimator, the design-detection provenance, optional sensitivity output, and an `audit_report`. The standalone `sp.audit(result)` call produces the same missing-evidence checklist for any fitted result and points to follow-up functions such as RD density tests, Honest-DiD sensitivity, or unobserved-confounding bounds. Each check carries one of four statuses — passed, failed, missing, or not applicable — and the last is decided from the fit’s own metadata by an explicit applicability rule (an over-identification test is not applicable to a just-identified IV model, and says why), so an agent is never sent after a test that does not exist. The companion `sp.validation_scope` answers the neighbouring question of which validation artifacts cover the fitted configuration and which of its outputs they compared (Section 5.4).

3.4. The bibliography verifier

Every public-facing citation emitted by the package must come from the curated `paper.bib`. Function specs store bibliography keys, result objects expose `cite(format=...)`, and tests

assert that registered keys parse cleanly and do not collide on author/year/title triples. When a key is missing, the package raises an error rather than emitting a plausible-looking citation. This targets a concrete agent failure mode: hallucinated references.

3.5. Comparison with existing platforms and prior art

Generating a JSON schema from a typed signature is solved by **Pydantic**, **FastAPI**, and the LLM-vendor tool adapters; what is package-specific here is that the schema is the statistical registry itself, tied to validation status and typed statistical errors, with result handles for estimate-to-audit-to-sensitivity chains – a validation-tiered tool catalogue with a citation verifier, which statistical help systems do not expose.

4. Worked examples

The paper uses seven worked examples as an end-to-end smoke test of the validated core rather than as seven miniature empirical papers. Each has an executable script under **Paper-JSS/replication/** that writes the JSON, table fragments, and figures the build reads. Together they cover selection on observables, IV, staggered DiD, RD, synthetic control, machine-learning effect estimation, matching, causal time-series counterfactuals, and—across all three ecosystems—panel counterfactual and interaction-effect estimators run on the same bytes in **Python**, **R**, and **Stata**. The staggered-DiD example is the main case study; it carries one fitted object from estimation through aggregation, sensitivity, and export. The examples also enforce the paper’s central boundary: original-data rows are claimed only where the public extract can be redistributed or regenerated from a reference package, and calibrated replicas are described as structural guards on sign, scale, and workflow shape. Table 2 maps those examples to their scripts and headline validation facts.

4.1. Card (1995): returns to schooling

The Card workflow ([Card 1995](#)) loads the NLSYM extract, fits OLS, 2SLS, DML, and a causal forest through the same `(y, X, data)` contract, and exports the resulting table with `sp.regtable`. `sp.datasets.card_1995()` returns the public `wooldridge::card` extract itself, so the session of Listing 1 runs on the original bytes; Listing 3 continues that session at a prompt and prints what it produces.

Example	StatsPAI surface exercised	exer-	Headline validation fact	Replication artifact
Card (1995)	<code>sp.regress</code> , <code>sp.dml</code> , forest	<code>sp.iv</code>	OLS and 2SLS match R on <code>wooldridge::card</code> at machine precision; DML/forest are reported as stochastic ML checks.	<code>ex01_card.py</code> ; original-data module 01_card
Basque (2003)	<code>sp.synth</code> , sensitivity		ADH special-predictor specification on the original bytes gives post-1970 gap -0.8946 vs R -0.8944 vs Stata -0.8943 ; default <i>V</i> -choice is disclosed separately.	<code>ex02_basque.py</code> ; original-data module 03_basque
<code>mpdta</code> staggered DiD	CS, aggregation, Honest-DiD, Bacon		Simple ATT matches <code>did</code> and <code>csdid</code> to machine precision on the calibrated replica bytes, and matches <code>did</code> at 2.3×10^{-14} on the original <code>did::mpdta</code> bytes.	<code>ex03_csdid.py</code> ; module 04_csdid
Lee (2008) RD	RD robust and density diagnostics		The native CCT bandwidth and robust estimate match <code>rdrobust</code> on the <code>RDsenate</code> extract to 3×10^{-13} ; the official <code>rdrobust</code> Python port agrees with the native path.	<code>ex04_lee.py</code> ; modules 06_rd, 09_rddensity
LaLonde / Dehejia-Wahba	OLS, PSM, IPW, balance diagnostics		Original public extracts from MatchIt and causalsens match R; the observational design preserves the classic sign/bias warning.	<code>ex05_lalonde.py</code> ; original-data modules 04, 04b
Advertising lift	<code>sp.causal_impact</code>		Known simulated lift = 4.2 is recovered at 4.306 with the 95% interval containing truth and pre-period RMSE below the registered guard.	<code>ex06_causal_impact.py</code>
fct / interflex	<code>sp.panel_view</code> , <code>sp.fct</code> , <code>sp.interflex</code>		Three fct outcome models and three interflex estimators return the same numbers as R <code>fct</code> / interflex and the authors' Stata commands on identical bytes (worst relative gap 10^{-9}).	<code>ex08_fct_interflex.py</code> ; modules 86_fct, 87_interflex

Table 2: Worked-example coverage and replication artifacts. Each row is reproduced by the named script under `Paper-JSS/replication/`; the main text keeps to the executable contract and the headline validation fact, while full per-row provenance lives in the replication archive.

```
>>> print(sp.regtable(ols, iv, dml, keep=["educ", "theta"], stars=True,
...                    model_labels=["OLS", "2SLS", "DML PLR"],
...                    rules="ascii"))
=====

```

	OLS	2SLS	DML PLR
educ	0.0740*** (0.0035)	0.1323*** (0.0492)	
theta			0.0741*** (0.0037)

```
=====
First-stage F
```

16.72

```

=====
N                3,010          3,010          3,010
R2                0.291          0.225
Adj. R2           0.289
F                204.932
=====
Standard errors in parentheses
*** p<0.01, ** p<0.05, * p<0.10
>>> print(sp.audit(iv))
Reviewer audit: IV-2SLS [iv]
1 of 2 applicable checks satisfied (passed 1, failed 0, missing 1; 1 n/a)

passed weak_instrument      value 16.72 vs threshold 10
n/a    overid_test          just-identified; no restriction to test
missing anderson_rubin_ci   run sp.anderson_rubin_ci

Missing checks are evidence a reviewer will expect, not errors.
n/a checks do not exist for this fit and are not counted.

```

Listing 3: The session of Listing 1, continued at a prompt: three result classes render into one table, and `sp.audit` separates the checks this fit has passed, still lacks, or cannot have. Printed with `rules="ascii"` so the transcript is byte-safe on any console code page.

The three linear-in-educ summaries land where the literature puts them: $\hat{\beta}_{OLS} = 0.07401$ against Card’s published 0.075, and $\hat{\beta}_{IV}^{nearc4} = 0.1323$ against his 0.132, reproducing the IV-above-OLS pattern that motivates the design; cross-fit DML agrees with OLS to the third decimal. R’s `lm+sandwich` and `AER::ivreg` return the same two coefficients on the same CSV bytes at relative errors 7.9×10^{-15} and 1.6×10^{-11} (Table 9). The `sp.audit` block is the audit-hook claim made concrete, and it distinguishes three states that a flat checklist conflates. The first-stage F of 16.72 clears the $F > 10$ rule of thumb (Staiger and Stock 1997; Stock and Yogo 2005); “passed” means only that this screen was met — it is neither a test of instrument validity nor a guarantee against weak-instrument distortion, which is why weak-instrument-robust Anderson–Rubin inference is still listed as missing. The over-identification test is reported as not applicable rather than missing: with one endogenous regressor and one excluded instrument the model is just-identified, the Sargan/Hansen statistic has zero degrees of freedom, and there is no restriction to test. The audit reads the instrument and endogenous-regressor counts from the fitted object to decide this; when an over-identified fit already carries a J statistic it is read and graded, and the text, JSON, and MCP outputs carry the same `not_applicable` status and reason.

The four estimators do not share one identification argument, and the table should not be read as four estimates of one parameter. The 2SLS coefficient uses `nearc4` as an excluded instrument and, with heterogeneous returns, identifies a local average effect for instrument-compliers under relevance, exclusion, independence, and monotonicity. The DML and forest calls in Listing 1 use no instrument: they adjust for the listed covariates and are causal only under conditional exogeneity of schooling, the assumption the IV design exists to avoid. On these data they therefore estimate an adjusted partial association, and the forest’s heterogeneity is heterogeneity in that association. The listing shows that the four fits share one result contract; it does not identify heterogeneous returns to schooling, which would require an IV-based heterogeneity estimator and its own validation. With that caveat, Listing 4 summarises the

fitted forest’s conditional effects.

```
>>> print(sp.cate_summary(cf).loc[["Mean", "Std. Dev.",
...                               "Q25", "Q75"]].round(4))
```

	CATE
Mean	0.0765
Std. Dev.	0.0164
Q25	0.0633
Q75	0.0859

Listing 4: How far the covariate-adjusted schooling–wage association varies across the sample, from the forest fitted in Listing 1. **Mean** is the plain average of the fitted conditional effects, not the forest’s doubly-robust average. With **grf**’s honest, 2,000-tree defaults the interquartile range spans roughly 6.3% to 8.6% per year around a mean of 7.7%, as **R grf** finds on the same bytes (6.3% to 8.7% around 7.6%): the estimated dispersion is modest, about a fifth of the 0.081 standard deviation the pre-1.29.0 forest engine reported for the same call on the same data.

The Track A fixture for this design is a calibrated replica, not the extract above, which is why its Card rows (Table 8) differ from the original-data rows of Table 9. Because schooling is continuous, the forest’s scalar aggregate is the average partial effect from the continuous-treatment debiased score, not the binary AIPW aggregate that Track B validates: `cf.average_treatment_effect()` returns 0.0787 (SE 0.0049), as does **grf** 2.6.1 on the same bytes—one draw from each engine, a stochastic screen rather than a grade—and both packages refuse the ATT and ATC targets here.

4.2. Abadie–Gardeazabal (2003): the Basque country

The Basque example (Abadie and Gardeazabal 2003) exercises the synthetic-control dispatcher and its sensitivity hooks, and it shows why a synthetic-control number must always travel with its specification and its data. Four combinations appear in this article; Table 3 lists them once, and the rest of the paper refers to them by row. Under the specification of Abadie *et al.* (2010) that `Synth::synth` implements—one special predictor per pre-treatment year and a nested search over the predictor weights V —the three implementations nearly coincide on the original bytes (row 1) and split on the calibrated replica (row 2), which is the paper’s one T4 disclosure (Section 5.4). The solver itself is certified where the answer is unique (row 4). The package default (row 3) uses the pre-treatment outcome path as the predictors, where V is not identified and is set to the identity; it is a documented convention with no reference row, and Figure 1 draws it.

4.3. Callaway–Sant’Anna staggered DiD: `mpdta`

The `mpdta` example is the main panel-workflow demonstration; Listing 5 shows the full fit-to-sensitivity chain from one fitted object.

`sp.callaway_santanna` estimates group-time ATTs, `sp.aggte` reports simple and dynamic aggregations, `sp.honest_did` applies the Rambachan–Roth sensitivity post-estimator, and `sp.bacon_decomposition` gives the TWFE weight diagnostic (Goodman-Bacon 2021). Listing 6 prints the two objects a reader of this literature would want to see—the aggregated effect, and how much parallel-trends violation the effect in the first treated period survives.

Specification	Data	Result	Where
1 ADH: special predictors, nested V	original Synth : :basque	post-1970 gap -0.8946 (StatsPAI), -0.8944 (R), -0.8943 (Stata); published -0.855	Table 9
2 ADH: special predictors, nested V	calibrated replica	StatsPAI within 4.2×10^{-4} of Stata ; R and Stata 2.3×10^{-2} apart (T4)	Module 07
3 package default: outcome path, $V = I$	calibrated replica	documented convention, no reference row	Figure 1
4 outcome path, unique solution	simulated, identified	weights 5.6×10^{-7} , gap 7.8×10^{-8} from Synth and synth (T2)	Module 52

Table 3: The four synthetic-control specification–data combinations used in this article. Rows 1–2 run the same specification on different bytes; row 4 certifies the solver on a problem with a unique answer.

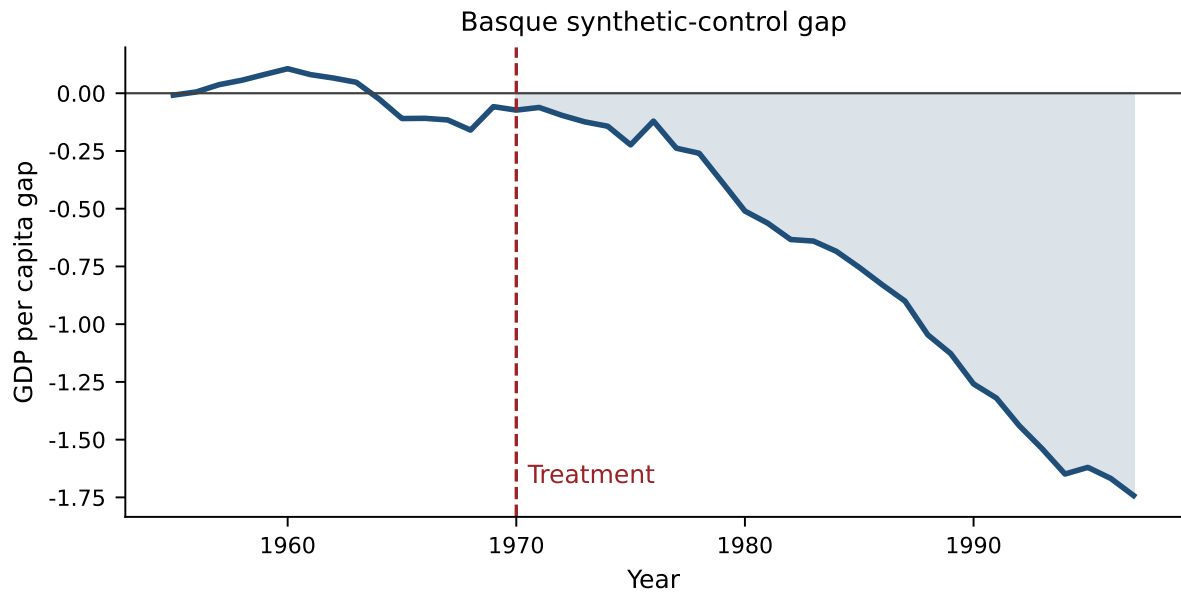


Figure 1: Synthetic-control gap (treated minus synthetic per-capita GDP) for the Basque country under the package default `sp.synth(method="classic")` (row 3 of Table 3), with the 1970 treatment marked. Regenerated by `replication/scripts/generate_figures.py`; numerical artifacts are written by `replication/scripts/ex02_basque.py`.

They are different estimands, and the transcript labels them as such.

Writing δ_t for the differential trend of treated and comparison units in period t (in years), the smoothness restriction of [Rambachan and Roth \(2023\)](#) is

$$\Delta^{SD}(M) = \{\delta : |(\delta_{t+1} - \delta_t) - (\delta_t - \delta_{t-1})| \leq M \text{ for all } t\}, \quad (1)$$

so M bounds how much the slope of the differential trend may change from one year to the next, in log points per year per year; $M = 0$ allows only a linear continuation of the pre-treatment trend. On the printed grid the first-period effect still excludes zero at $M = 0.0075$ and no longer at $M = 0.0112$; a continuous search (`sp.breakdown_m`) puts the breakdown value at 0.0080. That value, not the pre-trend test’s p -value, is the quantity a referee should be given, and it comes off the same fitted object as the point estimate.

```

mpdta = sp.datasets.mpdta()
cs = sp.callaway_santanna(mpdta, y="lemp", t="year", i="countyreal",
                          g="first_treat", control_group="nevertreated",
                          estimator="reg")
agg = sp.aggte(cs, type="simple", random_state=0)
es = sp.aggte(cs, type="dynamic", random_state=0)
hd = sp.honest_did(cs, e=0, method="smoothness")
bd = sp.bacon_decomposition(mpdta, y="lemp", treat="treat",
                           time="year", id="countyreal")

```

Listing 5: Callaway–Sant’Anna fit, simple and event-study aggregation, and Rambachan–Roth Honest-DiD bounds, all from the same fitted object; the Goodman–Bacon decomposition, a diagnostic of the two-way fixed-effects comparison weights, is computed from the data.

```

>>> print(f"simple ATT, all post periods: {float(agg.estimate):.6f}"
...       f" (SE {float(agg.se):.6f})")
simple ATT, all post periods: -0.032977 (SE 0.007961)
>>> print("ATT at e = 0, smoothness bounds:")
ATT at e = 0, smoothness bounds:
>>> print(hd.round(4).to_string(index=False))
   M  ci_lower  ci_upper  rejects_zero
0.0000  -0.0546  -0.0214           True
0.0037  -0.0566  -0.0129           True
0.0075  -0.0582  -0.0017           True
0.0112  -0.0601   0.0058          False
0.0149  -0.0632   0.0106          False
0.0224  -0.0706   0.0181          False

```

Listing 6: Two estimands from the fit of Listing 5: the simple aggregate over all post-treatment periods, and the effect at event time $e = 0$ with its Rambachan–Roth smoothness bounds. M bounds the change in slope of the differential trend between consecutive periods (Equation 1); `rejects_zero` is whether the robust confidence set still excludes zero at that bound.

That last clause is the case for the shared contract, and it can be measured. The Rambachan–Roth interval needs the *joint* covariance of the event-study coefficients, the relative times, and the position of the reference period. Composed by hand, the workflow extracts these from one package’s aggregation object and re-enters them into another’s sensitivity function. A careful user of **did** and **HonestDiD** passes the joint covariance, and nothing here says they would not; the point is that the hand-off offers a shortcut that runs without complaint, rebuilding the covariance from the reported standard errors. On `mpdta` the pre- and post-period coefficients are correlated up to 0.61, and the shortcut is not innocuous: with `diag(SE2)` in place of the joint covariance the breakdown value rises from 0.0080 to 0.0101, so the estimate would be reported as surviving trend deviations 26% larger than the data support, and the $M = 0$ interval shifts from $[-0.0546, -0.0214]$ to $[-0.0477, -0.0156]$. In **StatsPAI**, `sp.honest_did` reads the joint covariance, the relative times, and the aggregation weights from the fitted object through one extractor, `sp.event_study_vcov`, shared by nine event-study estimators; when a fit cannot supply a joint covariance it says so and falls back to a labelled approximation rather than fabricating one from standard errors. The single entry point also means a correctness fix

applies once: the 1.30.0 repair of the cohort-share term in that covariance (Table 18) reached every estimator’s sensitivity analysis at the same time. The saving is not lines of code; it is that the covariance, the reference period, and the estimand travel with the estimate instead of being re-typed. The `random_state=0` in Listing 5 is not decoration: `sp.aggte` follows `did` in defaulting to a 1,000-draw multiplier bootstrap, so the aggregated standard error is a random quantity and a printed transcript of it is only reproducible once the draw is pinned. The parity evidence sits on two data sets, and the paper keeps them apart. On the calibrated `mpdta` replica of Track A module 04, **StatsPAI**, `did`, and `csdid` all return the simple ATT -0.0329765 in the R-joined harness and agree within the registered iterative tolerance on the analytic SE. On the *original* `did::mpdta` bytes, **StatsPAI** and `did` return the simple ATT -0.03995 at relative error 2.3×10^{-14} and the vignette-printed overall dynamic ATT -0.0772 at 1.6×10^{-14} (Table 9). The homogeneous calibrated replica additionally checks that Callaway–Sant’Anna, Sun–Abraham (Sun and Abraham 2021), BJS/imputation (Borusyak, Jaravel, and Spiess 2024), and DR-DiD (Sant’Anna and Zhao 2020) collapse to the same estimand when their identification restrictions coincide. Figure 2 is the printed event-study counterpart to Listing 5.

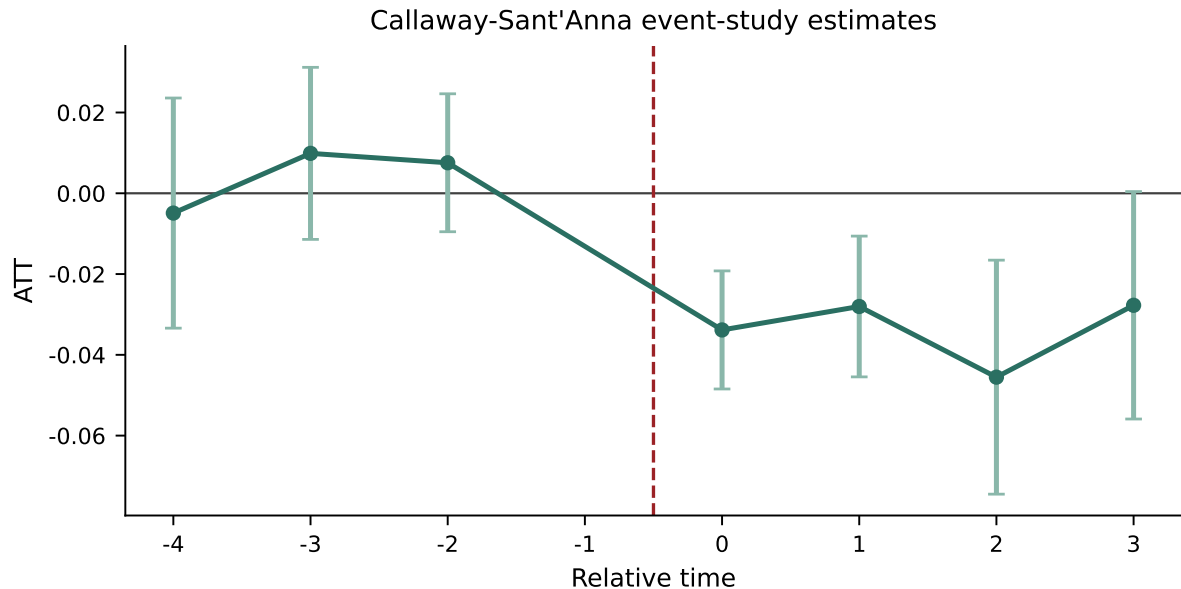


Figure 2: Callaway–Sant’Anna dynamic event study on `mpdta` (`sp.aggte(cs, type="dynamic")`): event-time ATTs with simultaneous confidence bands. Regenerated by `replication/scripts/generate_figures.py`; numerical artifacts are written by `replication/scripts/ex03_csdid.py`.

4.4. Regression discontinuity: the incumbency advantage (Lee-style sharp design)

The RD example deliberately separates the local-polynomial estimator from the bandwidth selector, and it keeps two data objects distinct to avoid over-claiming. The *exact* cross-language parity anchor is the public `rdrobust::RDsenate` extract — the U.S. Senate margin-of-victory sample distributed with `rdrobust`, a Lee-style sharp RD but *not* Lee (2008)’s original House data, which we do not redistribute. The calibrated `lee_2008_senate()` replica is

a redistribution-safe drift-guard, not a parity target: on it `sp.rdrobust` returns a robust jump of 0.0763, and the external test only asserts that the estimate stays in the $[0.05, 0.12]$ structural neighbourhood. On the RDsenate extract the native `sp.rdrobust` default, which implements the Calonico, Cattaneo, and Titiunik (2014) CCT bandwidth cascade and robust bias-corrected inference in StatsPAI itself (software article: Calonico, Cattaneo, and Titiunik 2015), returns the same MSE-optimal bandwidth (17.7544) and conventional and robust estimates as R’s default `rdrobust` at relative error below 3×10^{-13} (Section 5.3). Users who prefer the authors’ own code can call `sp.rdrobust(..., bwselect="cct")`, which delegates to the official `rdrobust` Python port; the two agree to 2×10^{-13} , and that agreement is recorded as a convergence check rather than counted as parity. The previous density-test selector gap is closed by the native `sp.rddensity` port (Section 5.3).

4.5. LaLonde (1986) / Dehejia–Wahba (1999): the NSW programme

The LaLonde (LaLonde 1986) / Dehejia–Wahba (Dehejia and Wahba 1999) workflow shows the same mature result surface under an experimental benchmark and an observational comparison. On `causalsens::lalonge.psid` (Blackwell 2014), the naive OLS ATT is $-\$15,200$ in both StatsPAI and R, and the covariate-adjusted OLS ATT is $\$751.9$ on both sides; on `MatchIt::lalonge`, 1:1 nearest-neighbour PSM agrees with `MatchIt` up to the registered caliper/default convention. The replication script also emits balance and overlap diagnostics: the point is not that matching fixes the benchmark, but that the platform keeps the experimental anchor, observational bias, and preprocessing diagnostics in one auditable pipeline.

4.6. Causal time series: advertising lift

The causal-impact example uses a deterministic Brodersen-style (Brodersen, Gallusser, Koehler, Remy, and Scott 2015) advertising-lift simulation rather than proprietary marketing data. The known post-intervention lift is 4.2 sales units per day; `sp.causal_impact` estimates 4.306 with SE 0.140, a 95% interval $[4.032, 4.579]$, cumulative effect 129.2 against true 126.0, and pre-period RMSE 0.562. These numbers are not cross-language parity claims; they are known-truth guards that the time-series counterfactual machinery, the result object, and the publication export path are wired correctly.

4.7. Three ecosystems, one number: panel counterfactuals and interaction effects

The last example shows the same estimator returning the same number in all three languages. It uses two method families whose authors ship both R and Stata implementations: the counterfactual estimators for time-series cross-sectional data of Liu, Wang, and Xu (2024) (`fect`: two-way fixed effects, interactive fixed effects, and matrix completion, fitted on the untreated cells) and the interaction-effect diagnostics of Hainmueller, Mummolo, and Xu (2019) (`interflex`: linear, binning, and kernel estimators of a conditional marginal effect). StatsPAI ports both natively, down to `fect`’s EM map and the linear-binning-plus-FFT density grid behind `interflex`’s adaptive kernel bandwidth. Listing 7 runs them on the panels of parity modules 86–87, after a `panelView`-style look at the treatment pattern (Mou, Liu, and Xu 2023).

```
fig, ax, info = sp.panel_view(panel, unit="id", time="time",
```

```

                                treat="D")
info["staggered"], info["has_reversals"]      # (True, False)
fe = sp.fect(panel, y="Y", treat="D", unit="id", time="time",
             covariates=["X1", "X2"], method="fe")
ife = sp.fect(panel, y="Y", treat="D", unit="id", time="time",
             covariates=["X1", "X2"], method="ife", r=2,
             vce="bootstrap", n_boot=200, seed=0)
mc = sp.fect(panel, y="Y", treat="D", unit="id", time="time",
             covariates=["X1", "X2"], method="mc", lam=0.002)
ife.detail                                    # ATT by relative period
bins = sp.interflex(sample, y="Y", d="D", x="X", z=["Z1"],
                    estimator="binning", cutoffs=[0.3, 1.7])
bins.model_info["tests"]["p_wald"]           # linear vs binning
kern = sp.interflex(sample, y="Y", d="D", x="X", z=["Z1"],
                    estimator="kernel", bw=1.0)

```

Listing 7: Panel counterfactuals and interaction effects. `sp.panel_view` reports adoption timing, reversals, and missing cells before any model is fitted; `sp.fect` imputes the untreated outcome of every treated cell under three outcome models; `sp.interflex` estimates how the treatment effect varies with a moderator and tests the linear-interaction restriction.

One scope boundary must be stated before the numbers: the R **fect** and **interflex** packages select the factor number r , the matrix-completion penalty λ , and the kernel bandwidth by cross-validation by default, and cross-validation draws random folds. The three-way parity claim below is therefore established at *user-specified* tuning values (the listing's `r=2`, `lam=0.002`, and `bw=1.0` are the values fixed by parity modules 86–87 so that all three implementations solve the same problem). Since 1.29.0 the selectors themselves are ported (`sp.fect(cv=True)`, `sp.interflex(cv=True)`). Their folds are random, so they cannot be T2, and the evidence is a stochastic screen rather than a T3 equivalence test. On identical folds the per-fold scores equal **fect**'s to 5.9×10^{-8} and **interflex**'s to 2.8×10^{-13} . Across fold seeds (twenty in R, five in **StatsPAI**) the **fect** selections coincide: $r = 2$ and the same λ in every run on both sides. For the **interflex** bandwidth R's own seeds do not agree—its twenty picks spread over six neighbouring grid points with no majority—so the check is only that every **StatsPAI** pick lies on R's flat stretch of the loss curve, which it does. The mean CV curves differ by less than four two-sample Monte Carlo standard errors at every grid point (`tests/reference_parity/test_fect_interflex_cv_parity.py`). The fit at a CV-selected value equals the fit at that value supplied by hand, so the T2 rows above are unchanged by the port.

Table 4 sets the **StatsPAI** numbers next to the R and Stata goldens; on the same CSV bytes the interactive-fixed-effects ATT agrees to 4×10^{-16} with R and 3×10^{-10} with Stata. The rows that are deliberately not three-way are as informative as the rest: the Stata **interflex** command uses a fixed Gaussian kernel and tests the linear restriction without the covariate-by-bin terms and against an F reference, and `sp.interflex` exposes both conventions (`adaptive=`, `wald_full_moderate=`, `wald_test=`) and reproduces each reference to machine precision. The complete row set (105 rows for **fect**, 25 for **interflex**) is in the parity ledger.

5. Numerical parity and statistical validity

Quantity	StatsPAI	R	Stata	rel(sp,R)	rel(sp,Stata)
<i>fect counterfactual estimators (module 86, staggered two-factor panel)</i>					
ATT, two-way FE (imputation)	2.500105	2.500105	2.500105	1.8×10^{-13}	9.8×10^{-10}
ATT, interactive FE, $r = 2$	2.241884	2.241884	2.241884	4.0×10^{-16}	2.8×10^{-10}
ATT, matrix completion, $\lambda = 0.002$	2.398529	2.398529	2.398529	2.8×10^{-15}	2.1×10^{-10}
<i>interflex marginal effects (module 87, binary treatment, moderator X, covariate Z)</i>					
linear ME at the grid midpoint	3.05408	3.05408	3.05408	5.8×10^{-16}	2.0×10^{-15}
binning effect, bin 1 (median $x = -0.27$)	1.51604	1.51604	1.51604	3.2×10^{-15}	5.9×10^{-16}
binning effect, bin 2 (median $x = 1.06$)	3.18212	3.18212	3.18212	2.8×10^{-16}	0
binning effect, bin 3 (median $x = 2.25$)	4.10838	4.10838	4.10838	1.1×10^{-15}	4.3×10^{-16}
Wald p-value, linear vs binning (R convention)	7.121×10^{-7}	7.121×10^{-7}	—	6.5×10^{-13}	—
Wald p-value (Stata convention: covariate-by-bin terms, F)	2.249×10^{-7}	—	2.249×10^{-7}	—	1.2×10^{-12}
kernel ME at the midpoint, adaptive bandwidth (R convention)	3.34108	3.34108	—	2.0×10^{-15}	—
kernel ME at the midpoint, fixed bandwidth (Stata convention)	3.27765	—	3.27765	—	1.1×10^{-15}

Table 4: Same bytes, three implementations. **StatsPAI** is re-estimated by `replication/scripts/ex08_fect_interflex.py`; the R (`fect 2.4.1`, `interflex 1.4.0`) and Stata (`fect_stata`, `SSC interflex`) columns are the frozen Track A goldens of modules 86 and 87. “—” marks a quantity the named reference does not report or computes under a different convention (shown on its own row).

The empirical claim of this paper is narrower than the package surface, and the census behind it is reported once, here. **StatsPAI** registers 1,255 public callables: 414 are **certified**, 144 **validated**, 694 **api_stable**, and 3 experimental. The two evidence tiers are earned in exactly two ways: **certified** by a comparison with a named external reference on identical bytes, **validated** by known-truth recovery or replication of published numbers. They answer different questions and are never summed in this paper, since a combined figure would let the second borrow the first’s authority. Coverage rows and documented disagreements (T4) are recorded against a function but earn nothing on their own. Both tiers and the parity grade are derived from one artifact, the committed parity index, through a single grade-to-tier function, and the tier additionally requires a stable API: 415 symbols have cross-language evidence and 414 are **certified** because `did_multilegt_dyn` is still experimental. Every **certified** symbol names its reference and tolerance (an R or Stata module for all but a handful whose canonical implementation is a Python package, enumerated in `PYTHON_REFERENCE_ROWS`), and an audit in the archive fails on any certified or validated symbol without an attached artifact.

The all-registered denominator dilutes coverage: 320 of the 1,255 callables are result or exception classes that can never carry a grade and 128 are infrastructure (tables, plots, schemas, data loading). Against the 807 estimator callables that remain, cross-language coverage is 415 (51.4%); `sp.parity_summary()` returns this split at call time. All 274 handwritten stable symbols carry parity, reference, or API-contract evidence; the remaining 601 stable auto-registered symbols are API-stable but not parity-backed, a denominator revisited in Section 9.

A label is only as good as the chain that attaches it to the evidence. Figure 3 lays out that chain as the package implements it; each link has its own check, and each check failed at least once on this package before it existed. The rest of this section reports the evidence and those checks in turn.

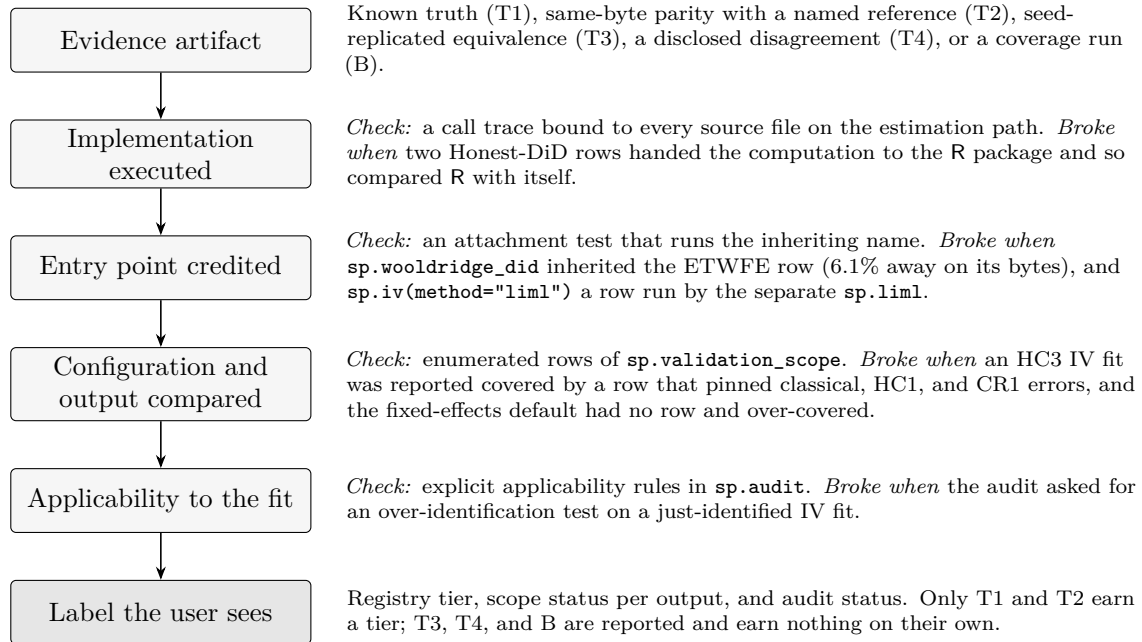


Figure 3: From an evidence artifact to the label a user or agent sees. Each link is checked mechanically; the right column names the check and the failure it caught in this package. Sections 5.3 (implementation), 5.8 (entry point), 5.4 (configuration and output), and 4.1 (applicability) give the details.

The validation design is a twelve-estimator suite with four evidence tracks: numerical parity (Track A), Monte Carlo coverage against known DGPs (Track B, separated into materialized B_{1000} rows and estimator-specific slow-suite B_s rows), computational scaling (Track C, Section 6), and agent-facing mechanical checks (Track D, Section 7). This is representative, not exhaustive: it covers the causal-inference core behind the paper’s statistical evidence claims, not every auxiliary helper in the registry.

Because the paper uses several graded vocabularies, Table 5 fixes them before any evidence is read. A **certified** or **validated** label is local to the named function and the evidence recorded for it; only a T2 row is a parity claim, and **api_stable** describes a stable signature, not statistical evidence.

5.1. Twelve-estimator validation suite

The reference implementations of Table 6 are cited once here: `lm` with **sandwich** (Zeileis *et al.* 2020), **AER**’s `ivreg` (Kleiber and Zeileis 2008), **fixest** (Bergé 2018), **did** (Callaway and Sant’Anna 2021), **rdrobust** (Calonico *et al.* 2014, 2015), **rddensity** (Cattaneo, Jansson, and Ma 2018, 2020), **Synth** (Abadie *et al.* 2011), **synthdid** (Arkhangelsky, Athey, Hirshberg, Imbens, and Wager 2021), **DoubleML** (Bach *et al.* 2022; Bach, Kurz, Chernozhukov, Spindler, and Klaassen 2024), **grf** (Athey, Tibshirani, and Wager 2019), and **MatchIt** (Ho *et al.* 2011), with the Stata commands `csdid` (Rios-Avila, Sant’Anna, Callaway, and Naqvi 2022), `sdid` (Clarke, PailaÑir, Athey, and Imbens 2024), `eventstudyinteract` (Sun and Abraham 2021), and `reghdfe`’s Poisson companion `ppmlhdfe` (Correia *et al.* 2020).

Vocabulary	Meaning
Registry tiers	certified / validated / api_stable / experimental : the per-function evidence label queryable at call time (Section 2.3).
Evidence grades T1–T5	Kind of numerical evidence behind a row: known-truth recovery (T1), same-byte cross-language parity (T2), seed-replicated equivalence of a stochastic estimator: both engines refitted under many seeds on fixed data and a two-one-sided test against a margin stated in sampling-SE units (T3), documented convention or identification boundary (T4), registered but not validated here (T5) (Section 5.4).
Stochastic screen (S)	A stochastic comparison held to a stated Monte Carlo tolerance (one seed per side, a handful of seeds, or one grid step) that is not an equivalence test. Reported, never graded T3.
Tracks A–D	The four evidence tracks of the validation suite: parity, coverage, performance, agent-facing mechanical checks.

Table 5: Terminology used by the validation sections. Each vocabulary is orthogonal: a **certified** function can carry T2 and T4 rows on different fixtures, and any row can be rebuilt on one or more reviewer paths.

#	Method	Entry point	Reference	Data / DGP	Tracks
1	OLS + HC SE	<code>sp.regress</code>	<code>lm+sandwich</code> ; <code>regress</code>	Card 1995	A,B,D
2	2SLS	<code>sp.iv</code>	AER ; <code>ivregress</code>	Card 1995	A,B,D
3	HDFE	<code>sp.fast.feols</code>	fixest ; <code>reghdfe</code>	HDFE panel	A,B,C,D
4	Callaway–Sant’Anna	<code>sp.callaway_santanna</code>	did ; <code>csdid</code>	<code>mpdta</code>	A,B,C,D
5	Sun–Abraham	<code>sp.sun_abraham</code>	fixest ; <code>eventstudyinteract</code>	<code>mpdta</code>	A,B,D
6	Bias-corrected RD	<code>sp.rdrobust</code>	rdrobust ; <code>rdrobust</code>	RDsenate	A,B,D
7	RD density test	<code>sp.rddensity</code>	rddensity ; <code>rddensity</code>	RDsenate	A,D
8	Classical SCM	<code>sp.synth</code>	Synth ; <code>synth</code>	Basque; unique DGP	A,C,D
9	Synthetic DID	<code>sp.sdid</code>	synthdid ; <code>sdid</code>	California-99	A,B,D
10	DML PLR	<code>sp.dml</code>	DoubleML	Card/401(k)	A,B,C,D
11	Causal forest	<code>sp.causal_forest</code>	grf	Clean-overlap DGP	A,B,D
12	PSM	<code>sp.psm</code>	MatchIt ; <code>teffects</code>	NSW-DW	A,D

Table 6: Twelve-estimator validation suite. In the Reference column the R package precedes the Stata command. Tracks are A (parity), B (materialized 1,000-draw coverage), C (performance), and D (agent-facing mechanical checks); row 8 dispatches as `sp.synth(method="classic")` and row 10 as `sp.dml(model="plr")`. Data marked with a named extract are calibrated replicas in Track A and original extracts in Table 9. Every coverage row runs through the member’s own entry point or one asserted bit-identical to it; supplemental Track-B rows (2x2 DiD, entropy balancing, the interactive DML model, `sp.panel`) are not members of this suite.

5.2. Estimands, identification, and inference contracts

Numerical agreement does not establish identification: two programs can compute the same quantity while both are inappropriate for a design that violates exogeneity, parallel trends,

continuity, or overlap. The contracts below are conditions on the user’s design and data, not properties a successful function call can certify.

For the linear rows, write $Y = X\beta + u$. The OLS coefficient is always a linear-projection parameter; interpreting it causally additionally requires the relevant conditional-mean restriction. With instruments Z , 2SLS solves the sample analogue of $E[Z(Y - X\beta)] = 0$; a LATE interpretation for a binary treatment further requires relevance, independence, exclusion, and monotonicity. Absorbing fixed effects changes the projection space, not these identification requirements. Track A pins HC1 or the named cluster; the classical default is pinned separately on the original Card extract (Section 5.4).

For staggered adoption, the target primitive is

$$\text{ATT}(g, t) = E[Y_t(1) - Y_t(0) \mid G = g],$$

identified under no anticipation, a stated never-treated or not-yet-treated comparison group, conditional parallel trends, overlap, and no interference. Callaway–Sant’Anna aggregates these group–time effects; Sun–Abraham first forms cohort-by-relative-time effects and then interaction weights. A joint pre-trend test is reported, but failure to reject is not proof of parallel trends and its power must be considered.

The RD row targets the sharp/fuzzy cutoff effect (or kink contrast) under continuity, local support, and no precise manipulation; its parity row uses the native robust bias-corrected interval at the native CCT MSE-optimal bandwidth (Section 5.3).

For RD density the null is equality of the one-sided density limits at the cutoff, a design diagnostic rather than an effect estimate. Classical SCM and SDID construct weighted untreated counterfactuals, so pre-treatment fit and the stability of the untreated outcome process are central, and their placebo distributions are design-based diagnostics, not i.i.d. sampling distributions.

DML PLR uses an orthogonal, cross-fitted score for the partially linear model $Y = \theta D + g(X) + \varepsilon$, $D = m(X) + v$. A causal reading of θ requires the corresponding conditional exogeneity and overlap conditions in addition to nuisance-rate conditions. The forest row targets $\tau(x) = E[Y(1) - Y(0) \mid X = x]$; its ATE/ATT summaries use an AIPW score and therefore require conditional exchangeability and overlap. PSM targets ATT by default and requires conditional exchangeability, common support, and a credible propensity specification; balance and overlap diagnostics remain part of the analysis even when the parity row passes.

5.3. Track A: Numerical parity

Track A uses three layers. First, estimators are tested against known DGP truths whenever a truth exists. Second, **StatsPAI** and the canonical R implementation read the same `%.16g` CSV bytes and write full-precision JSON. Third, when a directly comparable Stata command exists, the Stata side reads the same CSV and emits a sibling JSON artifact. The current R-joined harness has 89 modules. Of those, 85 carry a canonical or audited Stata reference; each of the 4 without one records a measured, command-level reason in `compare.py::STATA_SKIP_REASON`. Rows whose defaults differ for a documented reason are labelled as convention gaps rather than as ordinary passes. Table 7 summarizes the source-tree test layers behind these claims. Table 8 is a compact slice of the full ledger in Appendix A. Its fixtures are calibrated, redistribution-safe replicas; original public extracts are handled separately (Table 9). Of the

Layer	Harness	Tests	Role
Analytical/reference recovery	re-reference parity	4,475	Deterministic-DGP recovery, frozen R / Stata reference fixtures on identical bytes, and cross-estimator agreement; includes the 203 tests that pin Stata 's <code>vce()</code> grammar (27 estimators, 79 estimator-by-covariance cells at 10^{-6}) and its <code>test</code> , <code>lincom</code> and <code>margins</code> post-estimation commands
Published/replica parity	external parity	88	Pinned values for Card, Lee, Basque, NSW, <code>mpdta</code> , paper-level formulas, and the four DoubleML model-class pins
Monte Carlo coverage	coverage MC	32	Slow-marked headline DGP checks, robustness rows, and one smoke row

Table 7: Validation test layers available in the source tree used for this submission. Counts are the tests `pytest` collects in each harness, reported by `sp.validation_report(collect_tests=True)` and pinned as floors by `tests/test_jss_validation_api.py`; the Track A modules of Appendix A are a separate, module-level artifact.

89 modules, 88 receive a pass-type verdict, and the two non-T2 Track A rows stay visible: classical SCM on the calibrated Basque replica is a T4 identification/reference-disagreement disclosure, and the causal forest is a stochastic T3 equivalence claim (Section 5.4). Native `sp.sdidd` matches `synthdid`'s ATT at 2.6×10^{-15} and native `sp.rddensity` matches `rddensity`'s p -value at 3.3×10^{-11} .

Which implementation ran. A parity row is evidence about a **StatsPAI** algorithm only if the Python side ran one. A call-trace audit (`scripts/trace_parity_provenance.py`) runs each module under a profiler that records every call from **StatsPAI** into a package other than the numerical substrate and every subprocess it launches. The trace is bound to what it describes: it stores the SHA-256 of the module script, of every **StatsPAI** source file whose code ran while the module estimated, and of the committed result, with the versions of the packages it crossed into. A contract test fails if the registered classification and the trace disagree, if any of those files has changed since the trace, or if the trace meets a package outside the reviewed lists (classified `unclassified`, never native); a counterexample test edits one file on a module's estimation path, leaves the module script untouched, and requires the trace to go stale. Of the 89 modules, 86 run native code (`scikit-learn` appears only as a user-chosen nuisance learner) and 3 wrap a third-party library—panel fixed and random effects through `linearmodels`, ARIMA through `statsmodels`, fixed-effects GLMs through `pyfixest`—and are marked in Appendix A as evidence about the wrapper's conventions, not an independent algorithm. None calls the reference it is compared against. Before 1.31 two did: the Honest-DiD modules passed `backend="honestdid"`, which hands the computation to the R package, and the RD headline used the official `rdrobust` Python port. All three now run native code with evidence of their own. The same trace, run over the twelve original-data modules of Table 9, found one more: until 1.32.0 that ledger's RD row still called the port, so its agreement with

Estimator	Statistic	Data	StatsPAI	R ref.	Stata ref.	Max rel. err.	Verdict
<code>sp.fast.feols</code>	$\hat{\beta}_{x1}$	2-way HDFE, $N=10^4$	1.999503980	1.999503980	1.999503980	2.9×10^{-15}	T2; fixest ssc
<code>sp.regress</code>	$\hat{\beta}_{educ}$	Card replica	0.109986249	0.109986249	0.109986249	6.3×10^{-13}	T2
<code>sp.iv</code>	$\hat{\beta}_{educ}$	Card replica	0.141834409	0.141834409	0.141834409	1.1×10^{-11}	T2
<code>sp.callaway_santanna</code>	simple ATT	mpdta replica	-0.032976514	-0.032976514	-0.032976514	1.3×10^{-15}	T2
<code>sp.rdrobust</code>	robust RD, default CCT h	RDsenate	7.506502484	7.506502484	7.506502483	9.4×10^{-11}	T2; native, default h
<code>sp.dml</code>	$\hat{\theta}_{DML}$	Card replica	0.110228260	0.110228260	0.110228260	3.7×10^{-15}	T2; ddml bridge
<code>sp.causal_forest</code>	AIPW ATE	clean-overlap DGP	1.012009376	1.011741552	—	0.000265	one draw (S); T3 by seed study
<code>sp.psm</code>	ATT	NSW–DW replica	2277.690721	2277.690721	2277.690721	1.2×10^{-15}	T2; SE convention
<code>sp.synth (unique)</code>	avg post gap	identified SCM DGP	2.000000155	1.999999999	2.000000000	7.8×10^{-8}	T1+T2; identified DGP
<code>sp.synth (classic)</code>	avg post gap	Basque replica	-0.704244148	-0.687789210	-0.703950536	0.0239	T4; reference disagreement
<code>sp.frontier</code>	intercept	frontier DGP	2.071558153	2.071558166	2.071558168	7.2×10^{-9}	T2
<code>sp.decompose</code>	Oaxaca gap	Oaxaca DGP	0.221231026	0.221231026	0.221231026	6.3×10^{-16}	T2

Table 8: Representative Track A cross-language parity rows compiled from the same JSON artifacts as the supplementary parity ledger. Every row is inside its registered module-level tolerance in `tests/r_parity/compare.py`, which is 10^{-6} except where the verdict says otherwise: the DML row’s tighter 10^{-10} budget reflects its fully deterministic linear-learner pipeline, the forest row is a single draw from each engine whose interpretation rests on the seed-replicated comparison of Section 5.4, and the classical-SCM row is the 5×10^{-2} T4 disclosure. Data marked “replica” are the calibrated redistribution-safe fixtures; the Card-replica rows use the specification `lwage ~ educ + exper + expersq + black + south + smsa` with HC1 errors, which is why their coefficients differ from the shorter original-extract specification of Table 9.

R to 3.5×10^{-16} was the authors’ code compared with itself. It now runs the native selector and agrees to 3×10^{-14} . Seven of those twelve modules run native code throughout; the five NHEFS modules fit their propensity or outcome regressions with **statsmodels** (GLM and OLS) while the weighting, standardization, and inference are **StatsPAI**’s, and their rows are marked; none calls a port or a reference, and a contract test holds both ledgers to their traces. The native smoothness FLCI is compared with a documented modification of **HonestDiD** whose simulated folded-normal quantile is replaced by the exact one (the unmodified package sits 3×10^{-4} away, which is that simulation error), agreeing to 1.4×10^{-8} for $M > 0$; at $M = 0$, where the interval has a closed form (the GLS extrapolation of the pre-period coefficients), the native solver is within 2×10^{-9} of it and **HonestDiD**’s cone solver 5.7×10^{-6} . The native relative-magnitudes set, a port of the Andrews–Roth–Pakes conditional test (Andrews, Roth, and Pakes 2023), accepts the same points of **HonestDiD**’s specified finite θ grid on sixteen

cases—a statement about the grid, not the continuous set; its least-favourable hybrid, whose first stage is simulated, agrees within one grid step, a stochastic screen. The native RD selector returns **rdrobust**’s bandwidth and robust estimate on the RDsenate bytes to 3×10^{-13} .

Headline original-data rows. The complementary original-data ledger runs **StatsPAI** and the canonical R reference on the *original* public extracts shipped by R packages. It holds 12 modules and 35 statistic rows, including six epidemiology-side modules that reproduce the NHEFS worked examples of Hernán and Robins’ *What If* (IPW, g-formula, g-estimation, outcome regression, survival, and E-values). Table 9 shows the most reviewer-relevant cases; the full report is generated by `tests/orig_parity/compare_orig.py`, and the printed table is itself generated from those artifacts. The `did::mpdta` simple-ATT row makes the replica/original boundary concrete: the calibrated replica used by Track A module 04 returns -0.0330 on all three sides, while the original bytes return -0.03995 ; both are same-byte parity facts, and neither number is quoted against the other data set anywhere in this paper.

Extract	Statistic	Published	StatsPAI	R	Note
<code>wooldridge::card</code>	OLS educ	0.075	0.07401	0.07401	rel(sp,R) 7.9×10^{-15}
<code>wooldridge::card</code>	2SLS educ	0.132	0.1323	0.1323	rel(sp,R) 1.6×10^{-11}
<code>did::mpdta</code>	CS simple ATT	–	-0.03995	-0.03995	rel(sp,R) 2.3×10^{-14} ; no printed vignette anchor
<code>did::mpdta</code>	CS dynamic overall ATT	-0.0772	-0.07724	-0.07724	rel(sp,R) 1.6×10^{-14} ; vignette prints -0.0772
<code>Synth::basque</code>	ADH avg gap	-0.855	-0.8946	-0.8944	rel(sp,R) 1.8×10^{-4} ; Stata <code>synth</code> -0.8943 on the same bytes
<code>rdrobust::RDsenate</code>	robust RD jump	–	7.507	7.507	rel(sp,R) 3.2×10^{-14} ; native default selector
<code>causalsens::lalonge.psid</code>	adjusted OLS ATT	700	751.9	751.9	rel(sp,R) 2.2×10^{-13} ; public composite extract
NHEFS (<i>What If</i>)	IP-weighted ATE [‡]	3.4	3.441	3.441	rel(sp,R) 1.3×10^{-11}
NHEFS (<i>What If</i>)	g-formula ATE [‡]	3.5	3.463	3.463	rel(sp,R) 7.4×10^{-12}

Table 9: Selected original-data parity rows on public extracts, generated from `tests/orig_parity/results/`. All R references read the same CSV bytes as **StatsPAI**. The *Published* column is a heterogeneous anchor — an original-paper table value (Card, Basque), a printed-vignette value (the `did` dynamic ATT), a textbook program output (NHEFS), or a rounded composite-extract value (LaLonde) — shown for context, not as a parity target; the parity claim is the **StatsPAI**-versus-R pair in the *Note* column. The `did::mpdta` simple-ATT row has no printed vignette anchor, so its *Published* cell is empty by design. Every **StatsPAI** value comes from native code; rows marked [‡] fit their propensity or outcome regression with **statsmodels** (Section 5.3).

5.4. Evidence tiers and hard cases

Evidence is attached to configurations and outputs. A tier is attached to a function, but every artifact exercises one configuration—a method variant, an inference option, a data condition, a code path—and compares some outputs but not others. A parity row that pins the 2SLS coefficient and its classical, HC1, and CR1 standard errors says nothing about the HC3 standard error of the same coefficient. For the twelve suite estimators the package therefore records, per artifact, the values it actually ran on every dimension that changes the computation

(enumerated, never a wildcard) and the outputs it compared: the point estimate, the standard error, the coverage of the interval, or a diagnostic. A dimension is ignored only for an output that provably does not depend on it (the 2SLS point estimate does not depend on the covariance estimator), and the reason is stored with the exemption. `sp.validation_scope(result)` reads the configuration of a fitted object and reports, per output, the strongest matching evidence (reference, seed equivalence, stochastic screen, coverage simulation, disclosure, or none) and an overall status: `covered` when the point estimate and its standard error both have a T1 or T2 row, `estimate_only` when only the point estimate does, `stochastic_only`, `disclosure_only`, or `not_covered`. A dimension the fit did not record is listed as unchecked and never matches, and a queried value outside a dimension's domain is refused.

```
>>> fit = sp.iv("lwage ~ exper + expersq + black + south + smsa"
...           " + (educ ~ nearc4)", data=card, robust="hc3")
>>> scope = sp.validation_scope(fit)
>>> scope["status"], scope["outputs"]["se"]["status"]
('estimate_only', 'not_covered')
```

Listing 8: Output-level scope of the Card IV fit with HC3 errors. The Card reference test pins the coefficient and the classical, HC1, and CR1 errors, so the estimate is covered and the HC3 standard error is not.

Building the map at this resolution found defects in the package and in the map itself. `sp.iv(..., vce="cr2", cluster=...)` silently returned CR1 standard errors: the option fell through a keyword catch-all that the k-class fit ignored. It now reproduces `sp.ivreg`'s covariance bit for bit or raises for a method that cannot compute it. The fixed-effects member `sp.fast.feols` was pinned to `fixest` with `ssc="fixest"`, but its default charged absorbed effects that are nested in the cluster variable; a coverage row run through that entry point (the previous row ran `sp.panel`) found its CR1 errors 8% above the Monte Carlo SD, with coverage 0.979. The default now follows `fixest` (coverage 0.955, SE/SD 0.99). The map's LIML row cited a module that runs `sp.liml`, a separate code path from `sp.iv(method="liml")`; it now rests on a test that runs the latter on the same bytes. `sp.sun_abraham`'s default aggregate, the equal-weighted average of post-treatment event-time effects, had no reference row (module 05 pins the cohort-weighted aggregate); it now matches the same linear combination of `fixest`'s coefficients to 8×10^{-12} . An earlier pass of the map found `sp.iv`'s default classical covariance, printed in Listing 3, likewise unpinned; it is now held to `AER::ivreg` on the original Card extract at 8×10^{-11} . The map still has limits: it covers the twelve suite estimators only, and dimensions that no fitted object records (for example, the data condition) cannot be checked from a result.

Evidence grades. The grades are those of Table 5; T5 marks a function registered but not validated in this paper. Two rows need more than a tolerance to read correctly.

Causal forest: which Monte Carlo error? For a stochastic estimator two uncertainties must be kept apart: sampling variation across datasets, which the reported standard error of the ATE estimates, and the algorithm's own Monte Carlo variation across seeds on *fixed* data. "Agreement within combined Monte Carlo error" is a statement about the second, so it cannot be tested by dividing the gap between one **StatsPAI** fit and one **grf** fit by their sampling standard errors — which is how an earlier version of this row was graded. The

forest evidence now separates three tasks. *Truth recovery and inference* are Track B’s (the AIPW coverage row below). *Estimand agreement* holds by construction: both engines report the doubly-robust AIPW average (Athey et al. 2019), and given the same forest the AIPW operator agrees with **grf**’s to machine precision. *Algorithmic agreement* is tested by refitting each engine under 50 seeds (20 at 8,000 trees) on fixed data, at 500, 2,000, and 8,000 trees, on two datasets. The margin was chosen after inspecting the first, the reference-parity fixture, on an earlier engine, so that fixture is development data; the second, the Track A module-13 design, and the spaced-seed rerun of both were run after the margin was fixed. Seeds are spaced 10^5 apart on both sides, because **grf** forests grown from consecutive seeds share most of their random draws: a first run with consecutive R seeds understated **grf**’s seed-to-seed spread several-fold and made **StatsPAI**’s forest look several times noisier than it is. Table 10 reports the result. At every tree count on both datasets a two-one-sided test establishes equivalence within $\delta = 0.1$ sampling standard errors—a difference that moves a 95% interval endpoint by at most 5% of its half-width—with an upper bound of at most 0.049; with Welch–Satterthwaite t critical values in place of normal ones (20 seeds per side at 8,000 trees) the bound is unchanged to the third decimal. The engines’ seed-to-seed standard deviations are of the same order but not claimed equal: their ratio ranges from 0.79 to 1.48, and both fall with the number of trees, from about 13–14% of the sampling standard error at 500 trees to 6–7% at the default 2,000. The claim is equivalence, not identity: the engines are independent implementations, and at 8,000 trees, where Monte Carlo error is smallest, their seed means differ by up to 0.019 sampling standard errors. The single-fit Track A row is one draw from each distribution, a stochastic screen rather than a parity fact. Artifacts: `tests/reference_parity/test_grf_seed_mc_equivalence.py` and its fixtures.

Data	Trees	Seeds	Gap	TOST bound		Seed SD		Ratio	Equiv.
				z	Welch t	StatsPAI	grf		
Reference fixture	500	50/50	-0.004	0.049	0.049	0.136	0.138	0.99	yes
	2,000	50/50	+0.018	0.039	0.039	0.059	0.065	0.90	yes
	8,000	20/20	+0.019	0.032	0.033	0.031	0.021	1.48	yes
Module-13 design	500	50/50	+0.001	0.046	0.046	0.132	0.142	0.93	yes
	2,000	50/50	+0.004	0.026	0.026	0.062	0.068	0.90	yes
	8,000	20/20	-0.001	0.018	0.019	0.029	0.037	0.79	yes

Table 10: Seed-replicated comparison of `sp.causal_forest` and **grf** 2.6.1 on fixed data (AIPW ATE; ATT in the archive). Every column except Seeds and Ratio is in units of **grf**’s mean reported sampling standard error SE_s . “Gap” is the difference of seed means; the TOST bound is $|\text{gap}| + cSE_{MC}$ with c the one-sided 5% normal or Welch t critical value and SE_{MC} the gap’s algorithmic Monte Carlo standard error; “Seed SD” is each engine’s seed-to-seed standard deviation and “Ratio” their quotient. Equivalence against $\delta = 0.1SE_s$ requires both bounds below 0.1 and holds in every row. Generated by `replication/scripts/generate_forest_seed_table.py`.

Synthetic control: one specification, two data sets. The classical-SCM row is a T4 disclosure because the outer V optimization is non-convex and donor weights need not be unique, and how much that matters is data-dependent (Table 3). Under the same ADH specification the three implementations reconvene on the original `Synth::basque` bytes (pairwise within 4×10^{-4} , still above the 10^{-6} gate, consistent with solver tolerance on a non-convex problem)

but split on the calibrated replica, where R **Synth** and Stata **synth** themselves differ by 2.3×10^{-2} . A ledger that pooled the two facts into one tolerance would hide exactly the sensitivity a practitioner needs to know about. The independent evidence that the native solver is right where an answer exists is module 52: when the treated unit is exactly a unique convex combination of donors, the three implementations agree to the constrained solver’s tolerance.

Native **sp.augsynth** now ports **augsynth**’s Ridge+SCM convention (Ben-Michael, Feller, and Rothstein 2021) (ATT at 7.9×10^{-6} on the Basque fixture). Native **sp.gsynth** now ports **gsynth**’s two-way factor convention (Xu 2017), matching it to machine precision.

5.5. Track B: Monte Carlo coverage

Track B checks 95% interval coverage on known DGPs, and reports with each rate what is needed to explain it: the estimator’s bias, its Monte Carlo SD, the mean reported standard error, their ratio, and the interval length (Table 11). The artifact holds thirteen $B = 1,000$ rows. Nine correspond to suite members—OLS, 2SLS, **sp.fast.feols**, Callaway–Sant’Anna, Sun–Abraham, sharp RD, SDID, DML PLR, and the causal forest—and each runs through the member’s own entry point or one asserted bit-identical to it (**sp.ivreg** for **sp.iv**; **sp.causal_question** for the forest, whose estimate is **cf.average_treatment_effect()** exactly). The other four are supplemental: 2x2 DiD, entropy balancing, the interactive DML model, and fixed effects through **sp.panel**, which is family-level evidence and not credited to **sp.fast.feols**. Three suite members have no coverage row by construction: the RD density test is a diagnostic, classical SCM has no analytic interval, and PSM’s inference evidence is the Abadie–Imbens SE parity row.

Ten of the thirteen rates lie in the binomial acceptance region $[0.931, 0.967]$ of the table caption, and in all ten the mean standard error is within 4% of the Monte Carlo SD. Three fall outside it: SDID just below (0.928), the interactive DML model just above (0.968; ratio 1.04, mildly conservative), and DML PLR well below (0.883). Sharp RD (0.934) is inside the region but at its lower edge and over-rejects at the null (below), so it is examined with SDID. These rows are explained rather than graded. For *sharp RD* and *SDID* the mean standard error is within 2% of the Monte Carlo SD, so the shortfall is not one of average scale; the tails of the studentized statistic, the randomness of the standard error and its dependence on the estimate can still lower coverage, and the ratio alone does not say which. For RD the question can be settled directly: on the first 200 draws R **rdrobust** 4.0.0 returns the same intervals as **StatsPAI** to 6×10^{-12} and covers in exactly the same draws (**tests/coverage_monte_carlo/mechanisms/rd_drawwise_reference.py**), so the shortfall belongs to the procedure on this design, not to the port. *DML PLR* under-covers clearly (0.883), and two things contribute: its bias is half a Monte Carlo SD, and its mean standard error is 12% below the Monte Carlo SD (ratio 0.88). **sp.dml** returns **DoubleML**’s numbers to 10^{-16} on the same folds and learners, which rules out a porting difference; it does not make the interval adequate, and the package says so where a user will see it: the default-learner PLR configuration is reported by **sp.validation_scope** as **stochastic_only**, with this row and its 0.883 attached, and the limitation is in **sp.dml**’s documentation. Varying one thing at a time on the same design (**tests/coverage_monte_carlo/mechanisms/dml_plr_learners.py**, $B = 500$) locates it in the nuisance fit: with the true nuisance functions coverage is 0.952 and the bias vanishes; with a random forest or a spline Lasso it is 0.940 and 0.938; with the default

gradient boosting it is 0.892 at $n = 500$ and 0.936 at $n = 2,000$, where the bias is still 0.41 SD. DML’s orthogonal score makes nuisance error second order, not zero, and the default learner’s regularisation bias on this design shrinks too slowly to be negligible at these sample sizes. These experiments point to nuisance estimation as the main source of both the bias and the small standard error; they do not show that the default learner is inappropriate elsewhere. *Entropy balancing* over-covered at 1.000 in the previous revision because its standard error held the weights fixed and was twice the Monte Carlo SD; with the M-estimation variance it is now 0.945 with a ratio of 0.97 (Section 5.9). The *causal-forest* row, recomputed on the forest’s own doubly-robust average (Section 5.8; it previously measured a separate AIPW estimator and over-covered at 0.977), covers 0.959 with a ratio of 1.04 at the default 2,000 trees; a 300-draw check at 300 trees gives 0.973 (`tests/coverage_monte_carlo/mechanisms/forest_trees.py`). Table 12 records stress cases as failure-mode evidence rather than calibration results.

The size/power sweep closes a weakness of coverage-only reporting: an interval that never rejects can cover well without being useful. At the tested null, 2x2 DiD is conservative (size 0.024) and sharp RD rejects slightly too often (0.076 at $B = 500$), consistent with its coverage; the other rows are within Monte Carlo error of 0.05, including entropy balancing, whose size was 0.000 under the old standard error. Each curve is monotone on its own effect grid and reaches at least 0.90 at the largest effect; effect grids and DGPs differ, so this is not a cross-estimator ranking. DML, the forest, and SDID have no packaged power sweep, and the paper makes no power claim for them. Full rejection probabilities are in `tests/coverage_monte_carlo/results_b1000/size_power_b1000.json`.

5.6. Further checks

Estimators that should collapse to the same estimand on a given DGP are held to each other: Callaway–Sant’Anna, Sun–Abraham, multi-period DiD, and BJS imputation must agree on a homogeneous staggered design within a multiple of $\sqrt{SE_1^2 + SE_2^2}$. Two estimators on the same data are positively correlated, so this is a loose screen for gross disagreement, not an equivalence test, but it catches a shared bug that one-reference parity cannot. High- N fixtures tighten selected known-truth checks to two standard errors at $N = 5,000$ and 8,000; Honest-DiD is pinned to the closed-form breakdown value of [Rambachan and Roth \(2023\)](#). Event-study *paths* need a different test, because implementations differ in what each half of a path is differenced against ([Roth 2026](#)); `sp.event_study_convention()` records that choice per estimator. Running it found that `sp.did_imputation` documented Stata’s `pretrends(k)` but shipped leads equal to the symmetric benchmark times N_0/N ([Li and Strezhnev 2025](#)), so a violated assumption read as satisfied; the default now reproduces Stata’s leads and their standard errors to 1.2×10^{-12} and 8×10^{-13} .

5.7. Adjudicating a disagreement

A parity ledger is only as credible as its procedure for the rows that do *not* agree, so the procedure is fixed in advance and every registered tolerance carries the outcome. A disagreement on identical bytes is assumed to be a **StatsPAI** defect until the first diverging intermediate quantity – design matrix, weights, residuals, sandwich meat – has been located. If the divergence is a documented choice on both sides (a degrees-of-freedom divisor, a small-sample cluster correction, an analytic versus influence-function variance), the default follows the method’s own authors, the alternative is exposed as a keyword, and the gap is registered

Estimator	DGP	Cover.	Bias	MC SD	Mean SE	SE/SD	CI len.
<code>sp.regress</code> (HC1)	RCT with covariates	0.952	-0.0028	0.0707	0.0708	1.00	0.277
<code>sp.regress</code> 2x2 DiD	2-period	0.955	+0.0017	0.0974	0.0999	1.03	0.392
	homogeneous DiD						
<code>sp.ivreg</code> (HC1)	strong binary-Z IV	0.962	-0.0041	0.3687	0.3627	0.98	1.422
<code>sp.callaway_santanna</code>	homogeneous	0.947	-0.0007	0.1080	0.1090	1.01	0.427
	staggered ATT						
<code>sp.sun_abraham</code>	homogeneous	0.950	-0.0007	0.1214	0.1234	1.02	0.484
	staggered ATT						
<code>sp.panel</code> two-way FE	known-coefficient FE	0.948	-0.0024	0.1245	0.1233	0.99	0.486
	panel						
<code>sp.fast.feols</code> (CR1)	same FE panel, same	0.955	-0.0057	0.1026	0.1013	0.99	0.407
	draws						
<code>sp.rdrobust</code> sharp	known-jump RD,	0.934	-0.0061	0.1222	0.1201	0.98	0.471
	robust CI						
<code>sp.sdid</code> (placebo SE)	factor-model panel, 1	0.928	+0.0003	0.3095	0.3071	0.99	1.204
	treated						
<code>sp.ebalance</code>	CIA, ATT	0.945	-0.0014	0.0799	0.0774	0.97	0.303
	(M-estimation SE)						
<code>sp.dml</code> (IRM)	binary- D CIA, IRM	0.968	-0.0074	0.1582	0.1653	1.04	0.648
	ATE						
<code>sp.dml</code> (PLR)	continuous- D PLR,	0.883	-0.0261	0.0533	0.0472	0.88	0.185
	default learners						
<code>sp.causal_forest</code> (AIPW)	binary- D CIA,	0.959	+0.0095	0.0934	0.0968	1.04	0.379
	population ATE						

Table 11: Track B materialized coverage artifacts, all at $B = 1,000$. Next to the coverage rate: the estimator’s bias, its Monte Carlo SD, the mean reported SE, their ratio (SE calibration: 1 is exact, above 1 conservative), and the mean interval length. A calibrated 95% interval covers in Binomial(1,000, 0.95) of the draws, whose central 99% region is [0.931, 0.967]; rates outside it are discussed in the text. `sp.ivreg` returns bit-identical estimates and SEs to the suite’s `sp.iv` on this configuration (asserted in `test_validation_entry_points.py`). The HDFE member is measured through its own entry point, `sp.fast.feols`, on the same draws as the `sp.panel` row; the latter is family-level evidence only. The two DML rows are distinct configurations: the PLR row is the suite member; the IRM row (binary treatment, through `sp.causal_question`) is reported separately and does not stand in for it. The forest row reports the fitted forest’s own doubly-robust ATE at the default 2,000 trees.

with its mechanism and bounded by the tolerance. If the reference itself is the problem or the solution is not unique, the row is graded T4 and must carry independent evidence (a second reference, an analytic identity, or a uniquely identified DGP). If exact agreement is impossible by construction (forest randomization, bootstrap), the row is graded T3 only if both engines are refitted on fixed data under many seeds and pass an equivalence test against a margin stated in sampling-SE units; a single draw per side, a handful of seeds, or a Monte Carlo tolerance is reported as a stochastic screen and earns no grade. A gap that cannot be located is not called parity: it is graded “unexplained” in the tolerance audit and reported here as an open item. Two contract tests enforce the procedure mechanically: every R-joined standard-error row of every passing module must sit inside its module’s registered budget, and every Stata standard-error row that does not must be registered with a reconstruction of

Estimator	Stressor	Coverage	Interpretation
<code>sp.ivreg</code>	weak instrument, median $F \approx 2.5$ ($B = 1,000$)	0.882	Expected under-coverage; preflight warns and suggests LIML/AR.
<code>sp.callaway_santanna</code>	heterogeneous timing and magnitude ($B = 1,000$)	0.946	Post-fix influence-function scaling remains near nominal.
<code>sp.causal_forest</code>	severe overlap loss ($B = 300$)	0.900	Under-covers as propensities approach 0 and 1; audit flags overlap before interpretation.

Table 12: Track B robustness rows. These are descriptive failure-mode checks, not pass/fail calibration claims.

the Stata number. In release 1.32.0 all 89 modules pass the first test, and the second holds two entries, each reproduced from **StatsPAI**’s own quantities: the group-average SE of Stata `csdid` (the fixed-cohort-share aggregation of the joint influence functions, 6×10^{-15} ; module 04) and the PLIV degrees-of-freedom factor $N/(N - K)$ of Stata’s `ddml` (module 71). The Sun–Abraham row is the procedure’s worked example: it was carried as an open 0.7–2.2% item across two releases rather than absorbed into a wider tolerance, and the bisection that closed it in 1.24.0 found a defect on the **StatsPAI** side (Table 18).

5.8. Auditing the attachment, not only the number

A parity harness checks that two implementations produce the same number. It does not check the claim that sits one level above: that the function a reader calls is the function the harness ran. Track A modules are written against one entry point, while a package of this size exposes dispatchers, thin wrappers and convenience names that are *said* to reach the same estimator core. Every such name inherits a grade it did not earn on its own, and inheritance is where an unfalsifiable claim can hide – the numbers in the ledger stay correct while the set of functions they vouch for quietly grows.

Auditing those attachments in release 1.26.0 found that, of eleven registered alias attachments, seven cited a module that never called the aliased name. Each surviving attachment now names a test that runs the alias against the canonical entry point on the module’s own bytes. One did not survive: `sp.wooldridge_did`, credited with the ETWFE module as an alias of `sp.etwfe`, is a different estimator (its ATT is 6.1% away on that module’s bytes) and was demoted to the grade its own evidence supports. The configuration map of Section 5.4 found two more: its LIML row cited a module that runs `sp.liml`, not `sp.iv(method="liml")`, and its 2SLS rows mixed tests of `sp.iv` and `sp.ivreg`; both are now backed by tests that run the credited entry point. The failure mode also reaches coverage rows: Track B’s causal-forest row ran through `sp.causal_question()`, whose forest design fitted the forest and then reported a separate cross-fit AIPW estimator, so refitting with 30, 300, and 2,000 trees gave bit-identical results. It now reports the forest’s own doubly-robust average, identical to `cf.average_treatment_effect()`, and the row was recomputed.

5.9. Correctness history and residual risk

Every output-changing fix is pinned by a regression test and listed, with its version, in

Appendix B. Two entries explain the design better than the rest. The 1.23.0 fix shows what a reference comparison catches that unit tests do not: `sp.did(..., weights=)` never forwarded the weights to a staggered branch and silently returned the unweighted estimand, which only `did::att_gt(weightsname=)` on identical bytes exposed. The 1.25.0 fix shows the limit of a parity ledger: the causal-forest aggregation was wrong for a *continuous* treatment, a path no Track A or Track B row exercised, and it surfaced in a worked example. A ledger validates the configurations it runs and is silent about the rest.

The current release controls that residual risk in three ways rather than by pointing to the number of past fixes. First, the evidence for a fitted configuration is queryable: for any fit of the core, `sp.validation_scope` reports which artifacts ran its variant, inference option, and code path and which outputs they compared, and reports “not covered” rather than inferring coverage from a neighbouring configuration (Section 5.4). Second, the release is frozen: the paper describes tag `v1.32.0`, the archive carries the source snapshot and every artifact hash, and changes after the tag cannot alter a number printed here. Third, every recorded artifact can be rebuilt from its entry point (Table 16).

6. Computational performance

This section reports the measured Track C benchmark for the four estimators marked C in Table 6: high-dimensional fixed effects (`sp.fast.feols`), Callaway–Sant’Anna staggered DiD (`sp.callaway_santanna`), classical synthetic control (`sp.synth(method="classic")`), and double machine learning (`sp.dml(model="plr")`). The goal is not to declare universal speed superiority over the reference implementations. It is to measure, on identical inputs and tasks, whether the unified Python stack imposes a cost on routine work, to identify where the reference remains faster, and to make the comparison reproducible from the source tree.

6.1. Benchmark protocol

A speed ratio compares two implementations only if both perform the same task on the same input, so the protocol fixes both before anything is timed. Every input is drawn once, in Python (`tests/perf/_data.py`), written as CSV at round-trip precision, and read by *both* sides; each side records the SHA-256 of the file it read, and the comparator refuses a row whose two sides read different bytes or come from different runs. (Seeding **NumPy** and **R** with the same integer does not give the same data, which is how an earlier version of this benchmark compared different samples.) Outside the timed call each side records its target quantity, so the table can show that the two solved the same problem. Both sides run on one thread (**fixest**, **data.table**, BLAS, and **Numba** pinned), under one run identifier, on one machine (Apple Silicon arm64, 8 physical cores, 24 GB, macOS 26), with **StatsPAI** 1.32.0; each result file records the package versions, the source commit, and the 1-minute load average at the start. Each timed step starts only after five idle minutes, and is discarded and rerun if the load, sampled every 15 seconds, rises above 2.5 while it runs. Times are medians over repetitions after a warmup, reported with their interquartile range.

Estimator	Common task	Ref.	Max. N	StatsPAI (s)	Ref. (s)	Faster	Agree
HDFE	OLS, two absorbed FE, fixest iid SE		1,000,000	0.120 (0.001)	0.074 (0.001)	ref. 1.6 $\times$	2e-14
CS-DiD	ATT(g,t), pre-test, simple and dynamic aggregation	did	125,000	0.028 (0.000)	0.443 (0.002)	sp 15.8 $\times$	4e-13
SCM	ADH special predictors, nested V , no placebos	Synth	100	264.108 (0.498)	4.353 (0.108)	ref. 61 $\times$	0.000
DML	PLR, linear learners, 5 folds	DoubleML	10,000	0.0077 (0.0000)	0.0085 (0.0000)	tie 1.10 $\times$	0.00 SE

Table 13: Track C at the largest measured size (Apple Silicon arm64, one thread on both sides, **StatsPAI** 1.32.0). Both sides read the same input file (SHA-256 checked) and run the task shown; times are median seconds with the interquartile range in parentheses. “Agree” compares the target quantity outside the timed call: relative difference of the coefficient or simple ATT (HDFE, CS), absolute difference of the average post-treatment gap between the two non-convex SCM solvers, and the DML difference in units of its SE (the folds are drawn independently on each side).

HDFE. OLS of y on two regressors with firm and year effects absorbed and iid standard errors, $N \in \{10^4, 10^5, 10^6\}$: `sp.fast.feols` against `fixest::feols` (Bergé 2018).

CS-DiD. A balanced staggered panel of five periods with cohorts first treated in periods 2, 3, and 4 (the same calendar on both sides), $N_{\text{units}} \in \{1,000, 5,000, 25,000\}$: group-time ATTs by outcome regression against the never-treated, analytic standard errors (no multiplier bootstrap or uniform band on either side), the joint pre-test, and the simple and dynamic aggregations. `sp.callaway_santanna` returns all four in one call; the reference runs `did::att_gt` and two `did::aggte` calls.

SCM. A two-factor, 30-period panel with $n_{\text{donors}} \in \{20, 50, 100\}$ and the specification of Abadie *et al.* (2010): one outcome special predictor per pre-treatment year and a nested search over V , weights and gap only, no placebos, for `sp.synth(method="classic")` and **Synth::synth**. The two solvers differ by design (below), and each runs at its defaults for this specification. The package’s *default* call is a different task and is reported separately.

DML. Partially linear DML with linear nuisance learners and five folds, public-workflow construction included, $N \in \{1,000, 5,000, 10,000\}$: `sp.dml(model="plr")` against **DoubleML** Python in the same process, so that language-runtime overhead does not dominate a comparison of milliseconds.

6.2. Measured performance results

Table 13 reports the largest size of each benchmark and Figure 4 all measured sizes. In every row the two sides agree on the target quantity: to 2×10^{-14} for the HDFE coefficient, 4×10^{-13} for the simple ATT, 5×10^{-5} for the SCM gap (two non-convex solvers landing on the same solution), and within a hundredth of a standard error for DML, whose folds are drawn independently on each side.

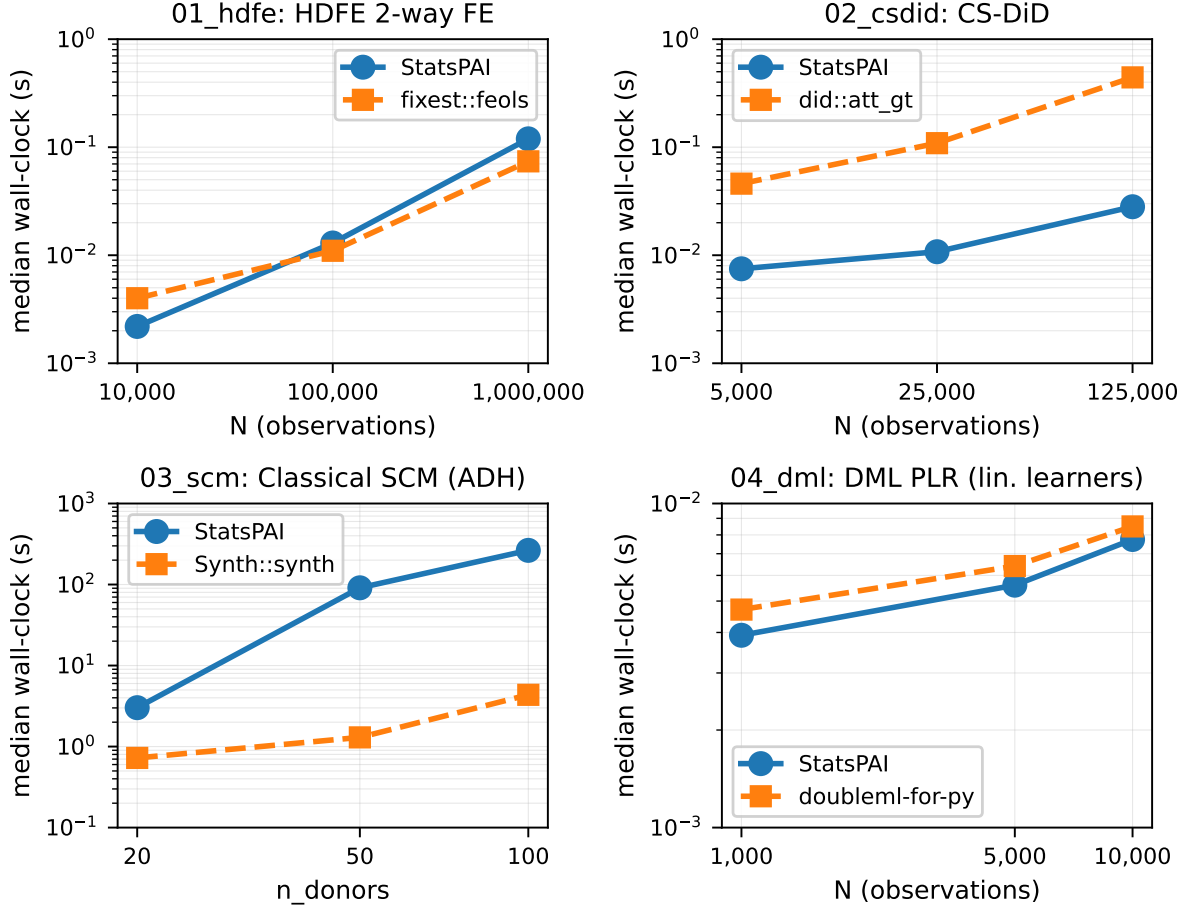


Figure 4: Median wall-clock time of **StatsPAI** and the reference at every measured size (log–log axes), on the shared inputs and settings of Table 13. Three sizes per benchmark show whether a gap is stable or growing across the measured range; they are not enough to establish an asymptotic order. Regenerated by `tests/perf/compare_perf.py`.

- **HDFE.** **StatsPAI** is faster at $N = 10^4$ (0.002 against 0.004 seconds), **fixest** is slightly faster at 10^5 (0.011 against 0.013), and **fixest** is $1.6\times$ faster at 10^6 (0.074 against 0.120 seconds), with both on one thread.
- **Callaway–Sant’Anna.** **StatsPAI** is faster at every size, by $6.1\times$ at 5,000 observations and $15.8\times$ at 125,000 (0.028 against 0.443 seconds); the gap widens with size, and we have not profiled the reference to say why. The output check caught an error in the previous version of this benchmark: its R leg stored the cohort as an integer column, into which **did** cannot write the **Inf** it uses for never-treated units, so **did** silently dropped them and used the last cohort as the comparison group. That leg timed a smaller, different problem.
- **Classical SCM.** On the common ADH task **Synth** is much faster: $4.2\times$ at 20 donors and $60.7\times$ at 100 (4.4 against 264 seconds). The cost is in the nested V search as **StatsPAI** runs it: six starts (equal, regression-based, four random) of a Nelder–Mead outer search, each outer evaluation solving the inner weight problem by SLSQP because donors outnumber the 15 pre-treatment predictors; profiling at 50 donors attributes 99% of the time to about

5,000 inner solves of 19 milliseconds. **Synth** runs one BFGS search from its regression-based start. The six starts are there to detect non-unique solutions (Section 5.4), so a single-start comparison would time a different procedure; a faster exact inner solver for the rank-deficient case is the main engineering target this study exposes. The package `default sp.synth(method="classic")`, which uses the outcome path with $V = I$ and runs in-space placebos over every donor, takes 7.5 seconds at 100 donors, of which 0.06 seconds is the fit and the rest the 100 placebo refits; it has no counterpart in the reference call and is not in the table.

- **DML PLR.** With linear learners the two Python implementations are within 10–20% of each other at every size ($1.10\times$ at $N = 10,000$: 0.0077 against 0.0085 seconds), and the interquartile ranges over 15 repetitions do not overlap, so **StatsPAI**’s wrapper adds no cost here; nothing is claimed about flexible learners, whose fitting time dominates both.

Interpretation. Two of four comparisons favour the reference at the largest size and one of them, classical SCM under the ADH specification, by a wide margin. The study supports a narrow claim: on the shared tasks measured here the unified interface adds no material overhead to HDFE, Callaway–Sant’Anna, or DML, and the nested- V synthetic-control solver is the clear performance gap. It does not support a general “Python is faster” claim, and the SCM row is kept in the table because it is negative evidence the package’s users need.

7. Agent-facing interface checks

This section reports only the *mechanical* and *contractual* evidence that the agent-facing surface is correctly wired up, which is not a behavioural claim. Whether the interface improves an LLM agent’s empirical work is a question that schema coverage cannot answer (Section 7.3).

7.1. Mechanical evidence

The function-schema layer is statically inspectable. We report in Table 14 both the function-level coverage (1,255 of 1,255 public functions emit a non-empty OpenAI-style JSON Schema; 422 currently expose curated or explicitly inherited agent metadata cards) and the parameter-level quality of those schemas (9,697 documented parameters across the 1,255 schemas, of which 35.0% carry an explicit default value and 4.9% carry enumerated choices). Parameter quality matters because the headline “1,255/1,255 emit a schema” figure is high by construction – the registry pipes *any* typed signature into a JSON Schema – whereas the stronger contract is that every parameter carries a non-empty description. Table 14 is an internal completeness audit, not a ranking: **statsmodels**, **linearmodels**, and **scikit-learn** ship no package-distributed, package-wide OpenAI-style tool-schema catalogues because they target different interfaces, and they do not rule out external wrappers.

A non-empty description is still only mechanical completeness; it does not make a parameter’s meaning unambiguous to a machine. For the eleven registered entry points of the validation suite, 24 of 143 schema parameters declare an enumerated domain and 55 a default. Mutually exclusive options (**robust=** and **vce=**), and conditions under which an option applies (a clustered covariance needs a cluster variable), are stated in the descriptions and enforced at call time by typed errors rather than expressed in the schema. Encoding them there, for the core first, is the next step for this interface.

Audit item	Count	Missing	Share
Public functions/classes	1,255	—	100.0%
Emit non-empty OpenAI-style tool schema	1,255	0	100.0%
Have at least one typed parameter	1,221	—	—
Have non-empty function description	1,255	0	100.0%
Have curated or inherited agent metadata	422	833	33.6%
Parameters across all schemas	9,697	—	100.0%
with non-empty description	9,697	0	100.0%
with explicit default value	3,395	—	35.0%
with enumerated choices (<code>enum</code>)	477	—	4.9%

Table 14: Internal mechanical schema audit for **StatsPAI** 1.32.0. “Agent metadata” means at least one non-empty assumption, pre-condition, failure mode, limitation, or typical/minimum sample-size field after explicit inheritance. The remaining schemas still emit name, signature, and description fields, but are not counted as curated. Of those, 413 are result or exception classes and infrastructure helpers (Section 5), where planning metadata has little to say; the other 420 are estimator entry points outside the curated set. Counts are regenerated from the live registry; this table makes no cross-package quality claim.

The same registry is exported as a versioned offline schema bundle, so non-Python clients can discover the tool surface without importing **StatsPAI**, and a coverage ratchet fails continuous integration if any agent-card counter falls.

7.2. Contractual trace evidence

The MCP server (Section 3.2) negotiates protocol revisions 2024-11-05, 2025-03-26, and 2025-06-18; the package’s own tests (not an external conformance certification) exercise the handshake, `tools/list` and `tools/call`, `isError` envelopes, and that MCP discovery sees the same registry as `sp.function_schema()`.

For this submission we also execute one deterministic end-to-end trace through the live MCP server. The script `replication/scripts/ex07_agent_trace.py` writes a temporary `mpdta.csv` (the calibrated replica of Section 4.3, so the traced estimate -0.0330 is the replica value, not the original-data -0.03995), calls `tools/list`, fits `callaway_santanna` with `as_handle=true`, sends the returned handle to `audit_result`, and then sends the same handle to `honest_did_from_result`. It finally resolves the returned citation keys through the MCP `bibtex` tool against the curated `paper.bib`, so the trace exercises the citation anti-hallucination contract rather than merely reporting key strings. The same trace also sends a stale result-handle call to `audit_result` and checks that the MCP response is a recoverable `isError` envelope with an actionable refit hint, so failure recovery is exercised rather than only described. Table 15 summarizes the trace; the normalized transcript is included in the replication archive. The trace verifies the agent-specific claim that the estimate $\rightarrow$ audit $\rightarrow$ sensitivity chain is executable without ferrying arrays or hand-copying citations through the model context.

7.3. No behavioural benchmark claim

Mechanical and contractual evidence does not answer whether an LLM agent using this

Trace item	Value
MCP tools listed	579
Estimate	-0.0330 (SE 0.0078)
Audit checks	4
Honest-DiD max rejecting M	0.0075
Recoverable stale-handle error	isError=True; hint present
Verified citation keys	callaway2021difference, rambachan2023more
BibTeX entries resolved	2

Table 15: Deterministic MCP trace used for the agent-facing interface check, including one recoverable stale-handle error path. Ephemeral result handles are normalized in the transcript shipped with the replication archive (`replication/results/ex07_agent_trace.txt`).

interface produces *better empirical analyses* than one using **statsmodels** or R packages through a Jupyter MCP shim. That question requires frozen model releases, fixed prompts, matched tool budgets, and a scoring rubric, and belongs in a separate behavioural study, with [Patil, Zhang, Wang, and Gonzalez \(2023\)](#) and [Liang et al. \(2023\)](#) as methodological precedents for tool-use and holistic-LLM benchmarks. To make that deferral auditable rather than vague, the archive includes `replication/results/agent_benchmark_protocol.md`, which records the matched arms, task families, scoring dimensions, leakage controls, and minimum report items required before any behavioural claim is made. The JSS contribution here is contractual: schemas, typed errors, result handles, citations, and the deterministic trace are reproducible interfaces, regenerated from the same validation ledger that governs human-facing calls rather than from a separate prompt-engineering layer, and they keep later benchmark scores from being confounded with missing metadata or unverifiable citations.

8. Computational details

StatsPAI installs from PyPI without compiled extensions of its own; its numerical core runs on the scientific Python stack (**numba**’s JIT compiler is the only compilation step, and it is optional at runtime). Every artifact of this article was regenerated in one environment, recorded exactly in `requirements-jss-lock.txt`: Python 3.10.20 with **NumPy** 2.2.6, **SciPy** 1.15.3, **pandas** 2.3.3, **statsmodels** 0.14.6, **scikit-learn** 1.6.1, **linearmodels** 7.0, **Numba** 0.65.1, and **matplotlib** 3.10.9 ([Hunter 2007](#)). `requirements-jss.txt` states the supported version ranges; it is not the lock. Optional heavy backends are isolated under extras and are lazily imported. The submitted numerical and timing claims do not require a compiled Rust extension, GPU, or commercial software.

The R parity harness is pinned by `tests/r_parity/renv.lock`. The reference environment uses R 4.5.2; the packages behind the twelve-estimator suite are **fixest** 0.14.0, **did** 2.3.0, **Synth** 1.1-10, **synthdid** 0.0.9, **rdrobust** 4.0.0, **rddensity** 2.6, **HonestDiD** 0.2.8, **DoubleML** 1.0.2, **grf** 2.6.1, and **MatchIt** 4.7.2, joined by **fect** 2.4.1 and **interflex** 1.4.0 for the three-ecosystem worked example; the complete dependency closure for all 89 modules (including **augsynth**, **gsynth**, **lme4**, **CBPS** ([Imai and Ratkovic 2014](#)), **ebal** ([Hainmueller 2012](#)), **optmatch** ([Hansen and Klopfer 2006](#)), **Matching** ([Sekhon 2011](#)), and **sbw** ([Wang and Zubizarreta 2020](#))) is recorded with exact versions and GitHub SHAs in the lockfile and per-row JSON provenance. The Stata bridge uses Stata 18 MP and records official/community command versions in

`tests/stata_parity/STATA_ENVIRONMENT.md`. All 85 R-joined Stata modules and all 89 R modules reproduce their frozen JSON outputs with zero drift; the Stata leg requires a licence, so the archive ships the frozen JSONs, matching do-files, and engine provenance, and the main paper remains auditable from the Python/R artifacts.

8.1. Distribution and installation

The package is distributed through PyPI (`pip install statspai==1.32.0`); this article describes release 1.32.0 (tag `v1.32.0`), and the replication archive carries the same source as a snapshot with its file hashes. Optional extras are exposed as `dev`, `performance`, `bayes`, `deepiv`, `neural`, `fixest`, `rd-cct`, and `plotting` bundles. The `rd-cct` extra installs the official **rdrobust** Python port (`rdrobust>=2.0`, maintained by the **rdrobust** authors), which backs the optional `bwselect="cct"` path of Section 4.4; that port is GPL-3 licensed, which is why it is an opt-in extra installed by the user rather than a bundled dependency of the MIT-licensed core. No parity claim in this paper depends on it: the native default selector reproduces its numbers on the audited fixture without it.

The test suite runs on Linux, macOS, and Windows in continuous integration; the acceptance evidence for this paper is the Track A–D artifacts, worked examples, and audit scripts, each regenerated rather than quoted.

8.2. Reproducibility of this paper

The evidence is rebuilt by one standalone script, `Paper-JSS/replication/reproduce.py`, and the script distinguishes two things a reader might mean by “reproduced”. Its default licence-free path (`-tier 1`; a few minutes on the laptop of Section 6, well inside the JSS one-hour threshold, while the longer recomputations are separate scripts) runs every generator the manuscript reads and then accounts for each of the twelve generated inputs of the compiled article, one by one, in `replication/results/reproduce_manifest.json`. Five are *recomputed* from the installed package on that run: the figures and tables of the worked examples, every printed listing and transcript (replayed statement by statement and compared line for line), the registry census behind every count in the text, and the MCP trace. Seven are *re-tabulated* from frozen experiment artifacts: the Track A and original-data parity tables, the Track B coverage table, the forest seed table, and the Track C table and figure. The frozen experiments have their own entry points (Table 16); each writes the artifact its table reads, so a recompute followed by Tier 1 rebuilds the tables from fresh numbers. A group whose reference side needs R fails when R is missing instead of reusing the frozen reference, so fresh Python numbers are never silently paired with old reference numbers; the timing comparator likewise refuses rows whose two sides read different input bytes or came from different runs. The *R path* (`-tier 2`) adds the pinned `renv` environment and re-executes every R reference of Track A on the committed bytes (module 81 needs the archived **DIDmultiplegt** 0.1.4, whose current release returns `NaN` on its own example, in a private library); since the parity claim is a comparison with R, auditing it at full strength requires this path. The *Stata path* (`-tier 3`) re-executes the migration bridges and requires a Stata licence.

Tier 1 was verified from a clean start: a fresh virtual environment built from the lock and the `v1.32.0` source, in a copy of the archive whose generated inputs had been deleted first (`-clean`), rebuilt all twelve. The Dockerfile pins the same environment—the Python base image by digest and every package by the lock—leaving only the TeX system packages that

typeset the PDF unpinned. Both sides of every parity row are also re-derivable on their own: `tests/r_parity/verify_reproduce.py` re-runs each R reference on the committed bytes and `tests/r_parity/verify_reproduce_py.py` does the same for the **StatsPAI** side, regenerating each fixture CSV byte for byte; all 89 modules clear a 10^{-9} reproduction floor with no exemptions.

Artifact	Tier 1	Full recomputation	Needs	Time
Worked examples, listings, census (Sec. 4)	recomputes	<code>reproduce.py</code> (Tier 1)	Python	a few min
Track A, StatsPAI side (Tab. 17)	re-tabulates	<code>-recompute parity-py</code> (and fixture bytes)	Python	~15 min
Track A, R side	re-tabulates	<code>-tier 2</code>	R 4.5.2, <code>renv.lock</code>	30–60 min
Track A, Stata side	re-tabulates	<code>-tier 3</code>	Stata 18	~30 min
Implementation trace	re-reads	<code>-recompute provenance</code>	Python	~15 min
Forest seeds (Tab. 10)	re-tabulates	<code>-recompute forest-seed</code> (both sides)	Python, R	~50 min
Track B and mechanisms (Sec. 5.5)	re-tabulates	<code>-recompute montecarlo</code>	Python; R for the RD draw-wise check	~2 h
Track C (Tab. 13)	re-tabulates	<code>-recompute performance</code> (both sides, one run)	Python, R, DoubleML	~1 h

Table 16: What the licence-free path recomputes and how to recompute the rest (options of `replication/reproduce.py`); each writes to the artifact path the paper reads. Dependencies beyond the lock for full recomputation are listed in `requirements-jss-recompute.txt` and `tests/r_parity/renv.lock`.

The replication archive contains the manuscript sources, replication scripts, selected package source, validation tests, R/Stata/reference parity artifacts, Monte Carlo and performance artifacts, and the deterministic MCP trace, and fits within the JSS attachment limit.

9. Discussion

The article’s claim is about what a validation label in a broad statistical package can be made to mean: attached to the implementation that ran, to the entry point that was tested, to the configuration and the output that were compared, and to checks that apply to the fit, with each attachment verified rather than asserted. The same design would serve any package that publishes evidence tiers.

The main limitations are as follows. First, 601 stable auto-registered symbols remain API-stable but not parity-backed (317 class-like, 284 function-like), and the other 93 symbols of the 694-symbol `api_stable` tier carry API, unit-contract, convention, or supplemental evidence only. The validated claim is limited to certified/validated entries and to the named rows and modules in the validation-suite evidence; the validation-evidence audit lists nine scoped limitation rows, each tied to a named function, evidence grade, and unsupported variant. Second, only one Track A row remains a methodological/T4 disclosure: classical SCM on the calibrated Basque replica, where reference implementations choose measurably different local optima; on the original `Synth:basque` bytes the same specification brings all three implementations within 4×10^{-4} (Section 5.4), so the disclosure is about data-dependent non-uniqueness, not a standing numerical defect. The causal-forest row is T3: the two forest engines are equivalent to within a tenth of a sampling standard error, with seed-to-seed spreads of the same order, which

is an equivalence claim, not parity. The 3 Track A rows served by third-party Python libraries validate **StatsPAI**'s wrappers of those libraries, not native algorithms. The configuration map behind `sp.validation_scope` covers the twelve suite estimators, and within them only the dimensions a fitted object records; for the rest of the registry the evidence notes name the artifact but not its configuration or the outputs it compared. Third, the **fect** and **interflex** cross-validation selectors for the factor number, penalty, and kernel bandwidth are ported in 1.29.0, but their folds are random, so the three-ecosystem parity of Section 4.7 is established at user-specified tuning values, and the selectors carry only given-fold score equality and a stochastic screen of the selection, not an equivalence test. Fourth, Stata reproduction requires a separate licence, so Stata rows are migration bridges rather than the licence-free core of the archive; 4 of the 89 modules carry no Stata reference, each with a measured reason. Fifth, Bayesian, causal-discovery, and behavioural-agent evaluations require different loss functions and are outside this software-validation article. Sixth, within DiD, **StatsPAI** implements the apparatus of the Baker *et al.* (2026) practitioner's guide but not the efficient combination of comparison groups and baseline periods (Chen, Sant'Anna, and Xie 2025) or selection-based pre-trend diagnostics (Ghanem, Sant'Anna, and Wüthrich 2026).

The unification itself has costs worth stating plainly, because they are the price of the design rather than defects of it. A unified API necessarily lags its upstream references: when **did** or **fixest** ships a new option, **StatsPAI** users wait for the ported, parity-tested version, and the validation regime that makes the port trustworthy is exactly what makes it slower to ship. The registry's own census quantifies a second cost: 601 of the registered symbols carry API-contract evidence only, so a user who strays from the validated core onto a long-tail helper is relying on software-engineering guarantees, not statistical validation – which is why the tier is printed at call time rather than in an appendix. Third, the evidence regime itself has a maintenance bill: every correctness fix in Table 18 exists because a parity or coverage artifact flagged it, and keeping 89 modules pinned against two external ecosystems is ongoing work that a single-method package does not carry.

A fourth cost is a limit of the regime rather than of the package. A parity ledger validates the paths it exercises and is silent about the rest: the 1.25.0 entry of Table 18, a continuous-treatment forest path no Track A or Track B row reaches, surfaced only in a worked example. That is why the registry records *which* artifact backs a label, why `sp.validation_scope` reports the configuration an artifact exercised, and why the worked examples are executed end-to-end.

The roadmap is to resolve or further bound the remaining classical-SCM reference-disagreement disclosure, extend $B = 1,000$ coverage beyond the headline rows, replace the inner solver of the nested synthetic-control search that dominates the performance gap of Section 6, and move the agent interface from mechanical conformance to scored behavioural benchmarking.

A final limitation is institutional: a large surface is maintained by a small team with one principal author. The risk is bounded by properties verifiable from the archive: the package is MIT-licensed with no relicensing clause, so the validated core can be forked; every output-changing fix is pinned by a regression test; and the cross-language parity harness is fully scripted and environment-pinned (`renv.lock`, recorded Stata versions, per-row JSON provenance), so any third party with the reference runtimes can re-execute it.

Author contributions

B. W. designed and implemented **StatsPAI**, the validation harnesses and registry API, and drafted the manuscript. S. R. supervised the applied-econometrics programme, advised on scope and validation priorities, and reviewed the manuscript. Both authors approved the final version.

Acknowledgements

The authors thank Stanford REAP for institutional support and feedback, and the maintainers of the Python, R, and Stata packages used as parity references. Any remaining errors are the authors' own.

Conflict of interest

StatsPAI is MIT-licensed open-source software. The authors are affiliated with StatsPAI Inc., which maintains the package; CoPaper.AI is a commercial downstream product that may call **StatsPAI** but is not required to use it. B. W. is the creator and principal maintainer. No external funder had a role in the validation studies, analysis, or manuscript preparation.

AI usage disclosure

Portions of code documentation and manuscript text were prepared with large-language-model assistance. The authors selected the claims, reviewed the text, and remain responsible for all statistical decisions. Numerical claims are tied to tests, replication scripts, frozen JSON artifacts, and cited references; citations are checked by the verifier in Section 3.4.

References

- Abadie A, Diamond A, Hainmueller J (2010). "Synthetic control methods for comparative case studies: Estimating the effect of California's tobacco control program." *Journal of the American Statistical Association*, **105**(490), 493–505. doi:10.1198/jasa.2009.ap08746.
- Abadie A, Diamond A, Hainmueller J (2011). "Synth: An R Package for Synthetic Control Methods in Comparative Case Studies." *Journal of Statistical Software*, **42**(13), 1–17. doi:10.18637/jss.v042.i13.
- Abadie A, Gardeazabal J (2003). "The Economic Costs of Conflict: A Case Study of the Basque Country." *American Economic Review*, **93**(1), 113–132. doi:10.1257/000282803321455188.
- Andrews I, Roth J, Pakes A (2023). "Inference for Linear Conditional Moment Inequalities." *Review of Economic Studies*, **90**(6), 2763–2791. doi:10.1093/restud/rdad004.
- Angrist JD, Pischke JS (2009). *Mostly Harmless Econometrics: An Empiricist's Companion*. Princeton University Press. ISBN 978-0-691-12035-5. doi:10.1515/9781400829828.

- Anthropic (2024). “Model Context Protocol Specification.” <https://modelcontextprotocol.io/>. Accessed 2026-09-04.
- Arkhangelsky D, Athey S, Hirshberg DA, Imbens GW, Wager S (2021). “Synthetic difference-in-differences.” *American Economic Review*, **111**(12), 4088–4118. doi:10.1257/aer.20190159.
- Athey S, Tibshirani J, Wager S (2019). “Generalized random forests.” *The Annals of Statistics*, **47**(2), 1148–1178. doi:10.1214/18-aos1709.
- Bach P, Chernozhukov V, Kurz MS, Spindler M (2022). “DoubleML – An Object-Oriented Implementation of Double Machine Learning in Python.” *Journal of Machine Learning Research*, **23**(53), 1–6. URL <https://jmlr.org/papers/v23/21-0862.html>.
- Bach P, Kurz MS, Chernozhukov V, Spindler M, Klaassen S (2024). “DoubleML: An Object-Oriented Implementation of Double Machine Learning in R.” *Journal of Statistical Software*, **108**(3), 1–56. doi:10.18637/jss.v108.i03.
- Baker A, Callaway B, Cunningham S, Goodman-Bacon A, Sant’Anna PHC (2026). “Difference-in-Differences Designs: A Practitioner’s Guide.” *Journal of Economic Literature*, **64**(2), 498–557. doi:10.1257/jel.20251650.
- Battocchi K, Dillon E, Hei M, Lewis G, Oka P, Oprescu M, Syrgkanis V (2019). “EconML: A Python Package for ML-Based Heterogeneous Treatment Effects Estimation.” URL <https://github.com/py-why/EconML>.
- Ben-Michael E, Feller A, Rothstein J (2021). “The Augmented Synthetic Control Method.” *Journal of the American Statistical Association*, **116**(536), 1789–1803. doi:10.1080/01621459.2021.1929245.
- Bergé L (2018). “Efficient estimation of maximum likelihood models with multiple fixed-effects: the R package FENmlm.” *Technical Report 18-13*, DEM Discussion Paper, Department of Economics, University of Luxembourg.
- Blackwell M (2014). “A Selection Bias Approach to Sensitivity Analysis for Causal Effects.” *Political Analysis*, **22**(2), 169–182. doi:10.1093/pan/mpt006.
- Borusyak K, Jaravel X, Spiess J (2024). “Revisiting event-study designs: Robust and efficient estimation.” *Review of Economic Studies*, **91**(6), 3253–3285. doi:10.1093/restud/rdae007.
- Brodersen KH, Gallusser F, Koehler J, Remy N, Scott SL (2015). “Inferring causal impact using Bayesian structural time-series models.” *The Annals of Applied Statistics*, **9**(1), 247–274. doi:10.1214/14-aos788.
- Callaway B, Sant’Anna PHC (2021). “Difference-in-differences with multiple time periods.” *Journal of Econometrics*, **225**(2), 200–230. doi:10.1016/j.jeconom.2020.12.001.
- Calonico S, Cattaneo MD, Titiunik R (2014). “Robust nonparametric confidence intervals for regression-discontinuity designs.” *Econometrica*, **82**(6), 2295–2326. doi:10.3982/ECTA11757.
- Calonico S, Cattaneo MD, Titiunik R (2015). “rdr robust: An R Package for Robust Non-parametric Inference in Regression-Discontinuity Designs.” *The R Journal*, **7**(1), 38–51. doi:10.32614/RJ-2015-004.

- Cameron AC, Gelbach JB, Miller DL (2008). “Bootstrap-Based Improvements for Inference with Clustered Errors.” *Review of Economics and Statistics*, **90**(3), 414–427. doi:10.1162/rest.90.3.414.
- Card D (1995). “Using Geographic Variation in College Proximity to Estimate the Return to Schooling.” In LN Christofides, EK Grant, R Swidinsky (eds.), *Aspects of Labour Market Behaviour: Essays in Honour of John Vanderkamp*, pp. 201–222. University of Toronto Press, Toronto. ISBN 0802028640.
- Cattaneo MD, Jansson M, Ma X (2018). “Manipulation Testing Based on Density Discontinuity.” *The Stata Journal*, **18**(1), 234–261. doi:10.1177/1536867X1801800115.
- Cattaneo MD, Jansson M, Ma X (2020). “Simple Local Polynomial Density Estimators.” *Journal of the American Statistical Association*, **115**(531), 1449–1455. doi:10.1080/01621459.2019.1635480.
- Chen H, Harinen T, Lee JY, Yung M, Zhao Z (2020). “CausalML: Python Package for Causal Machine Learning.” doi:10.48550/arXiv.2002.11631.
- Chen X, Sant’Anna PHC, Xie H (2025). “Efficient Difference-in-Differences and Event Study Estimators.” ArXiv preprint, first posted 2025-06-21, 2506.17729.
- Chernozhukov V, Chetverikov D, Demirer M, Duflo E, Hansen C, Newey W, Robins J (2018). “Double/debiased machine learning for treatment and structural parameters.” *The Econometrics Journal*, **21**(1), C1–C68. doi:10.1111/ectj.12097.
- Clarke D, Pailańir D, Athey S, Imbens G (2024). “On synthetic difference-in-differences and related estimation methods in Stata.” *The Stata Journal*, **24**(4), 557–598. doi:10.1177/1536867X241297914.
- Conley TG (1999). “GMM estimation with cross sectional dependence.” *Journal of Econometrics*, **92**(1), 1–45. doi:10.1016/s0304-4076(98)00084-0.
- Correia S, Guimarães P, Zylkin T (2020). “Fast Poisson estimation with high-dimensional fixed effects.” *Stata Journal*, **20**(1), 95–115. doi:10.1177/1536867X20909691.
- Dehejia RH, Wahba S (1999). “Causal Effects in Nonexperimental Studies: Reevaluating the Evaluation of Training Programs.” *Journal of the American Statistical Association*, **94**(448), 1053–1062. doi:10.1080/01621459.1999.10473858.
- Ghanem D, Sant’Anna PHC, Wüthrich K (2026). “Selection and Parallel Trends.” ArXiv preprint, v15 dated 2026-07-24 (first posted 2022-03-17), 2203.09001.
- Goodman-Bacon A (2021). “Difference-in-differences with variation in treatment timing.” *Journal of Econometrics*, **225**(2), 254–277. doi:10.1016/j.jeconom.2021.03.014.
- Hainmueller J (2012). “Entropy Balancing for Causal Effects: A Multivariate Reweighting Method to Produce Balanced Samples in Observational Studies.” *Political Analysis*, **20**(1), 25–46. doi:10.1093/pan/mpr025.
- Hainmueller J, Mummolo J, Xu Y (2019). “How Much Should We Trust Estimates from Multiplicative Interaction Models? Simple Tools to Improve Empirical Practice.” *Political Analysis*, **27**(2), 163–192. doi:10.1017/pan.2018.46.

- Hansen BB, Klopfer SO (2006). “Optimal Full Matching and Related Designs via Network Flows.” *Journal of Computational and Graphical Statistics*, **15**(3), 609–627. doi:10.1198/106186006X137047.
- Harris CR, Millman KJ, van der Walt SJ, Gommers R, Virtanen P, Cournapeau D, Wieser E, Taylor J, *et al.* (2020). “Array Programming with NumPy.” *Nature*, **585**(7825), 357–362. doi:10.1038/s41586-020-2649-2.
- Ho DE, Imai K, King G, Stuart EA (2011). “MatchIt: Nonparametric Preprocessing for Parametric Causal Inference.” *Journal of Statistical Software*, **42**(8), 1–28. doi:10.18637/jss.v042.i08.
- Hunter JD (2007). “Matplotlib: A 2D Graphics Environment.” *Computing in Science & Engineering*, **9**(3), 90–95. doi:10.1109/MCSE.2007.55.
- Imai K, Ratkovic M (2014). “Covariate Balancing Propensity Score.” *Journal of the Royal Statistical Society Series B*, **76**(1), 243–263. doi:10.1111/rssb.12027. Fong, Ratkovic & Imai are the CBPS R package maintainers (<https://github.com/kosukeimai/CBPS>).
- Imbens GW, Wooldridge JM (2009). “Recent Developments in the Econometrics of Program Evaluation.” *Journal of Economic Literature*, **47**(1), 5–86. doi:10.1257/jel.47.1.5.
- Kleibers C, Zeileis A (2008). *Applied Econometrics with R*. Springer, New York. doi:10.1007/978-0-387-77318-6.
- LaLonde RJ (1986). “Evaluating the Econometric Evaluations of Training Programs with Experimental Data.” *American Economic Review*, **76**(4), 604–620.
- Lam SK, Pitrou A, Seibert S (2015). “Numba: A LLVM-Based Python JIT Compiler.” In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, pp. 1–6. doi:10.1145/2833157.2833162.
- Lee DS (2008). “Randomized experiments from non-random selection in U.S. House elections.” *Journal of Econometrics*, **142**(2), 675–697. doi:10.1016/j.jeconom.2007.05.004.
- Li Z, Strezhnev A (2025). “Benchmarking Parallel Trends Violations in Regression Imputation Difference-in-Differences.” doi:10.31235/osf.io/ngr3d_v1. SocArXiv preprint, v1 posted 2025-07-11.
- Liang P, Bommasani R, Lee T, Tsipras D, Soylu D, Yasunaga M, Zhang Y, Narayanan D, Wu Y, Kumar A, Newman B, Yuan B, Yan B, Zhang C, Cosgrove C, Manning CD, Ré C, Acosta-Navas D, Hudson DA, Zelikman E, Durmus E, Ladhak F, Rong F, Ren H, Yao H, Wang J, Santhanam K, Orr L, Zheng L, Yuksekgonul M, Suzgun M, Kim N, Guha N, Chatterji N, Khattab O, Henderson P, Huang Q, Chi R, Xie SM, Santurkar S, Ganguli S, Hashimoto T, Icard T, Zhang T, Chaudhary V, Wang W, Li X, Mai Y, Zhang Y, Koreeda Y (2023). “Holistic Evaluation of Language Models.” *Transactions on Machine Learning Research*. 2211.09110.
- Liu L, Wang Y, Xu Y (2024). “A Practical Guide to Counterfactual Estimators for Causal Inference with Time-Series Cross-Sectional Data.” *American Journal of Political Science*, **68**(1), 160–176. doi:10.1111/ajps.12723.

- McKinney W (2010). “Data Structures for Statistical Computing in Python.” In *Proceedings of the 9th Python in Science Conference*, pp. 56–61. doi:10.25080/Majora-92bf1922-00a.
- Mou H, Liu L, Xu Y (2023). “Panel Data Visualization in R (panelView) and Stata (panelview).” *Journal of Statistical Software*, **107**(7), 1–20. doi:10.18637/jss.v107.i07.
- Patil SG, Zhang T, Wang X, Gonzalez JE (2023). “Gorilla: Large Language Model Connected with Massive APIs.” ArXiv preprint, first posted 2023-05-24, 2305.15334.
- Pedregosa F, Varoquaux G, Gramfort A, Michel V, Thirion B, Grisel O, Blondel M, Prettenhofer P, Weiss R, Dubourg V, Vanderplas J, Passos A, Cournapeau D, Brucher M, Perrot M, Duchesnay É (2011). “Scikit-learn: Machine Learning in Python.” *Journal of Machine Learning Research*, **12**, 2825–2830. URL <https://jmlr.org/papers/v12/pedregosa11a.html>.
- Pustejovsky JE, Tipton E (2018). “Small-Sample Methods for Cluster-Robust Variance Estimation and Hypothesis Testing in Fixed Effects Models.” *Journal of Business & Economic Statistics*, **36**(4), 672–683. doi:10.1080/07350015.2016.1247004.
- Rambachan A, Roth J (2023). “A more credible approach to parallel trends.” *Review of Economic Studies*, **90**(5), 2555–2591. doi:10.1093/restud/rdad018.
- Rios-Avila F, Sant’Anna PHC, Callaway B, Naqvi A (2022). “CSDID: Stata Module for the Estimation of Difference-in-Differences Models with Multiple Time Periods.” https://github.com/friosavila/csdid_drddid. Stata user-contributed package (GitHub); accessed 2026-09-04.
- Roodman D, Nielsen MØ, MacKinnon JG, Webb MD (2019). “Fast and wild: Bootstrap inference in Stata using boottest.” *The Stata Journal*, **19**(1), 4–60. doi:10.1177/1536867x19830877.
- Roth J (2026). “Interpreting event-studies from recent difference-in-differences methods.” *The Japanese Economic Review*, **77**(2), 275–288. doi:10.1007/s42973-026-00235-x. ArXiv:2401.12309.
- Roth J, Sant’Anna PHC, Bilinski A, Poe J (2023). “What’s trending in difference-in-differences? A synthesis of the recent econometrics literature.” *Journal of Econometrics*, **235**(2), 2218–2244. doi:10.1016/j.jeconom.2023.03.008.
- Sant’Anna PHC, Zhao J (2020). “Doubly Robust Difference-in-Differences Estimators.” *Journal of Econometrics*, **219**(1), 101–122. doi:10.1016/j.jeconom.2020.06.003.
- Seabold S, Perktold J (2010). “Statsmodels: Econometric and Statistical Modeling with Python.” In *Proceedings of the 9th Python in Science Conference*, pp. 92–96. doi:10.25080/Majora-92bf1922-011.
- Sekhon JS (2011). “Multivariate and Propensity Score Matching Software with Automated Balance Optimization: The Matching Package for R.” *Journal of Statistical Software*, **42**(7), 1–52. doi:10.18637/jss.v042.i07.
- Sharma A, Kiciman E (2020). “DoWhy: An End-to-End Library for Causal Inference.” doi:10.48550/arXiv.2011.04216.

- Staiger D, Stock JH (1997). “Instrumental Variables Regression with Weak Instruments.” *Econometrica*. doi:10.2307/2171753.
- Stock JH, Yogo M (2005). “Testing for Weak Instruments in Linear IV Regression.” *Identification and Inference for Econometric Models*. doi:10.1017/cbo9780511614491.006.
- Sun L, Abraham S (2021). “Estimating dynamic treatment effects in event studies with heterogeneous treatment effects.” *Journal of Econometrics*, **225**(2), 175–199. doi:10.1016/j.jeconom.2020.09.006.
- Virtanen P, Gommers R, Oliphant TE, Haberland M, Reddy T, Cournapeau D, Burovski E, Peterson P, *et al.* (2020). “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python.” *Nature Methods*, **17**(3), 261–272. doi:10.1038/s41592-019-0686-2.
- Wager S, Athey S (2018). “Estimation and inference of heterogeneous treatment effects using random forests.” *Journal of the American Statistical Association*, **113**(523), 1228–1242. doi:10.1080/01621459.2017.1319839.
- Wang B, Rozelle S (2026). “StatsPAI: A Unified, Agent-Native Python Toolkit for Causal Inference and Applied Econometrics.” *Journal of Open Source Software*, **11**(125), 10604. doi:10.21105/joss.10604. URL <https://doi.org/10.21105/joss.10604>.
- Wang Y, Zubizarreta JR (2020). “Minimal Dispersion Approximately Balancing Weights: Asymptotic Properties and Practical Considerations.” *Biometrika*, **107**(1), 93–105. doi:10.1093/biomet/asz050.
- Wing C, Yozwiak M, Hollingsworth A, Freedman S, Simon KI (2024). “Designing Difference-in-Difference Studies with Staggered Treatment Adoption: Key Concepts and Practical Guidelines.” *Annual Review of Public Health*, **45**, 485–505. doi:10.1146/annurev-publhealth-061022-050825.
- Xu Y (2017). “Generalized synthetic control method: Causal inference with interactive fixed effects models.” *Political Analysis*, **25**(1), 57–76. doi:10.1017/pan.2016.2.
- Zeileis A, Köll S, Graham N (2020). “Various Versatile Variances: An Object-Oriented Implementation of Clustered Covariances in R.” *Journal of Statistical Software*, **95**(1), 1–36. doi:10.18637/jss.v095.i01.

A. The complete Track A ledger

Table 8 in Section 5.3 prints a twelve-row slice of Track A, chosen to span the validation suite. The slice is a reading aid, not the evidence. The evidence is Table 17 below: every one of the 89 modules, with the worst residual gap each one carries against its canonical R reference and, for 85 of them, against a canonical or audited Stata bridge.

The difference columns are worst-case over each module’s headline rows and cover *point estimates only*; every SE row of every passing module is gated separately (Section 5.7). The parenthetical notes name the convention, delegation, or common specification under which a row is claimed, and italic text in the Stata column records a *measured* reason that no portable Stata artifact exists.

ID	Method	Worst diff vs R	Worst diff vs Stata	Verdict
01	OLS + HC1 SE	$\text{rel} \leq 1.1 \times 10^{-12}$	$\text{rel} \leq 1.3 \times 10^{-12}$	PASS
02	2SLS + HC1 SE	$\text{rel} \leq 1.1 \times 10^{-11}$	$\text{rel} \leq 1.1 \times 10^{-11}$	PASS
03	HDFE 2-way FE	$\text{rel} \leq 5.2 \times 10^{-15}$ (fixest/reghdfe small-sample correction)	$\text{rel} \leq 2.9 \times 10^{-15}$	PASS
04	CS-DiD simple ATT	$\text{rel} \leq 1.3 \times 10^{-15}$ (estimate and SE both $\text{rel} < 1 \times 10^{-9}$)	$\text{rel} \leq 1.3 \times 10^{-15}$	PASS
05	Sun–Abraham event study	$\text{rel} \leq 2.8 \times 10^{-11}$ (fixest agg='att' summary parity)	$\text{rel} \leq 2.7 \times 10^{-11}$	PASS
06	RD CCT bias-corrected	$\text{rel} \leq 5.7 \times 10^{-14}$ (native CCT default- <i>h</i> ; official Py port recorded separately)	$\text{rel} \leq 9.4 \times 10^{-11}$	PASS
07	Classical SCM	$\text{rel} \leq 0.024$ (T4 reference disagreement: native tracks Stata; R Synth differs; exact recovery on identified DGP 52)	$\text{rel} \leq 0.00042$	<i>GAP</i>
08	DML PLR (LinReg learners)	$\text{rel} \leq 0$ (shared explicit fold_id)	$\text{rel} \leq 3.7 \times 10^{-15}$	PASS
09	RD density (CJM)	$\text{rel} \leq 9.3 \times 10^{-12}$ (native CJM/rddensity default parity)	$\text{rel} \leq 1.8 \times 10^{-11}$	PASS
10	Honest DiD bounds	$\text{abs} \leq 1.41 \times 10^{-8}$ (native FLCI; vs HonestDiD with exact quantile, and closed form at M=0; shipped HonestDiD's simulated quantile shown separately)	$\text{abs} \leq 9.38 \times 10^{-9}$	PASS
11	PSM 1:1 NN	$\text{rel} \leq 1.2 \times 10^{-15}$ (se_teffects_ai joins Stata teffects psmatch's Abadie-Imbens 2016 estimated-score variance at $\text{rel} 7 \times 10^{-8}$ via the abadie_imbens_2016 SE option; the default psmatch2 ai(1) SE is a documented convention (sample vs population ATT, no score term))	$\text{rel} \leq 2 \times 10^{-16}$	PASS
12	Synthetic DID	$\text{rel} \leq 7.8 \times 10^{-16}$ (native Frank-Wolfe/zeta ATT parity; backend-native placebo SE diagnostics)	$\text{rel} \leq 7.2 \times 10^{-8}$	PASS
13	Causal forest (AIPW)	$\text{rel} \leq 0.00026$ (T3; single draw per engine, graded by seed-replicated equivalence within 0.1 sampling SE)	<i>bridge artifact not materialized: Stata 19's official cate command is the candidate causal-forest/AIPW reference, but the verified local runtime is Stata 18 and 'which cate' fails.</i>	PASS
14	OLS + cluster-robust SE	$\text{rel} \leq 3 \times 10^{-14}$	$\text{rel} \leq 2.2 \times 10^{-14}$	PASS
15	HDFE + cluster SE	$\text{rel} \leq 5.2 \times 10^{-15}$ (fixest/reghdfe nested-FE cluster correction)	$\text{rel} \leq 2.9 \times 10^{-15}$	PASS
16	BJS imputation	$\text{rel} \leq 4.8 \times 10^{-8}$ (R/Stata simple-ATT point parity; SE rows side-specific)	$\text{rel} \leq 3.5 \times 10^{-7}$	PASS
17	Wooldridge ETWFE	$\text{rel} \leq 1.8 \times 10^{-13}$ (emfx aggregation and cluster SE matched)	$\text{rel} \leq 3.9 \times 10^{-14}$	PASS

(continued on the next page)

(Table 17 continued)

ID	Method	Worst diff vs R	Worst diff vs Stata	Verdict
18	Augmented SCM	rel $\leq 7.9 \times 10^{-6}$ (native centered Ridge+SCM parity; backend='augsynth' is a migration bridge)	rel $\leq 1.7 \times 10^{-8}$	PASS
19	Generalized SCM (Xu 2017)	rel $\leq 7.7 \times 10^{-14}$ (native gsynth/fect two-way FE factor convention parity)	rel $\leq 2.2 \times 10^{-15}$	PASS
20	Goodman–Bacon decomposition	rel $\leq 5.6 \times 10^{-16}$ (R/Stata bacondecomp dyad parity)	rel $\leq 9.6 \times 10^{-9}$	PASS
21	Honest-DiD relative-mags	abs $\leq 4.44 \times 10^{-16}$ (native ARP conditional set; identical grid points)	abs $\leq 5.55 \times 10^{-17}$	PASS
22	sensemakr robustness	rel $\leq 5 \times 10^{-8}$ (sensemakr RV and benchmark bound-scale parity)	rel $\leq 5 \times 10^{-8}$	PASS
23	E-value (closed form)	rel $\leq 5.8 \times 10^{-14}$	rel $\leq 1.2 \times 10^{-16}$	PASS
24	Cox proportional hazards	rel $\leq 8.4 \times 10^{-16}$	rel $\leq 2.1 \times 10^{-10}$	PASS
25	Linear mixed model	rel $\leq 2.6 \times 10^{-12}$	rel $\leq 8.3 \times 10^{-11}$	PASS
26	GLMM logit (Laplace)	rel ≤ 0.00017	rel $\leq 1.4 \times 10^{-8}$	PASS
27	GLMM logit (AGHQ, n=8)	rel $\leq 1.7 \times 10^{-7}$ (same 8-point AGQ likelihood (logLik rel 1×10^{-13}); intercept within reference optimiser scatter on a flat objective)	rel $\leq 9.8 \times 10^{-7}$	PASS
28	Stochastic frontier (cross-sec.)	rel $\leq 4.1 \times 10^{-8}$	rel $\leq 4 \times 10^{-8}$	PASS
29	Panel SFA (Pitt–Lee)	rel $\leq 2.8 \times 10^{-6}$ (slope parity; intercept/sigma rows are scale diagnostics)	rel $\leq 1.8 \times 10^{-6}$	PASS
30	Blinder–Oaxaca decomposition	rel $\leq 6.3 \times 10^{-16}$ (twofold vs threefold split convention)	rel $\leq 1.3 \times 10^{-16}$	PASS
31	DFL reweighting	rel $\leq 1.2 \times 10^{-9}$	rel $\leq 1.8 \times 10^{-13}$	PASS
32	RIF / UQR (median)	rel $\leq 2.2 \times 10^{-15}$	rel $\leq 1.4 \times 10^{-16}$	PASS
33	VAR (vars::VAR)	rel $\leq 3.1 \times 10^{-15}$	rel $\leq 6.6 \times 10^{-15}$	PASS
34	Local projections	rel $\leq 5 \times 10^{-15}$ (lpirfs Cholesky/unit-shock path (py-R) plus the direct-OLS Jorda regression (3-way); SEs are not joined on the direct rows because sp.local_projections applies Newey–West while lm/regress report classical OLS)	rel $\leq 4.4 \times 10^{-15}$	PASS
35	Panel FE/RE + Hausman [†]	rel $\leq 4.7 \times 10^{-14}$ (FE/RE and plm-style Hausman parity; Stata sigmamore diagnostic row)	rel $\leq 1.5 \times 10^{-15}$	PASS
36	Causal mediation (IKT)	rel $\leq 6.7 \times 10^{-15}$	rel $\leq 3.6 \times 10^{-15}$	PASS
37	PPML + HDFE	rel $\leq 4.9 \times 10^{-13}$ (post PPML FE-score fix; robust SE within 0.5% of fixest)	rel $\leq 2.2 \times 10^{-15}$	PASS

(continued on the next page)

(Table 17 continued)

ID	Method	Worst diff vs R	Worst diff vs Stata	Verdict
38	DR-DID (SZ 2020)	rel $\leq 2.6 \times 10^{-15}$ (panel DR-DID calibrated propensity parity with R/Stata)	rel $\leq 2.2 \times 10^{-16}$	PASS
39	ARIMA(2,0,0) [†]	rel $\leq 7.4 \times 10^{-7}$ (stats::arima ML / tightly converged Stata arima)	rel $\leq 9.3 \times 10^{-9}$	PASS
40	Quantile reg (median)	rel $\leq 3.3 \times 10^{-15}$ (Hendricks-Koenker nid sandwich (1.32))	rel $\leq 4.4 \times 10^{-15}$	PASS
41	Tobit (left-censored)	rel $\leq 1.1 \times 10^{-13}$ (post observed-information Hessian fix)	rel $\leq 4.6 \times 10^{-12}$	PASS
42	Negative binomial	rel $\leq 5.1 \times 10^{-10}$	rel $\leq 1 \times 10^{-11}$	PASS
43	Heckman 2-step	rel $\leq 1 \times 10^{-11}$	rel $\leq 1 \times 10^{-11}$	PASS
44	Multinomial logit	rel $\leq 2.6 \times 10^{-7}$ (post observed-information Hessian fix)	rel $\leq 1.5 \times 10^{-11}$	PASS
45	Ordered logit	rel $\leq 1.5 \times 10^{-7}$ (post observed-information Hessian fix)	rel $\leq 3.4 \times 10^{-7}$	PASS
46	Conditional logit	rel $\leq 1.3 \times 10^{-8}$	rel $\leq 1.3 \times 10^{-8}$	PASS
47	PPML + 3-way HDFFE	rel $\leq 1.2 \times 10^{-15}$ (Post Gauss-Seidel multi-FE fix; sp matches at 1×10^{-15})	rel $\leq 1.2 \times 10^{-16}$	PASS
48	Binary probit	rel $\leq 3.1 \times 10^{-7}$	rel $\leq 1.6 \times 10^{-8}$	PASS
49	Ordered probit	rel $\leq 3.7 \times 10^{-7}$	rel $\leq 2.1 \times 10^{-10}$	PASS
50	Arellano-Bond GMM	rel $\leq 9 \times 10^{-16}$ (R/Stata dynamic-panel fixture; block-diagonal instruments and one-step GMM weights match)	rel $\leq 1.4 \times 10^{-15}$	PASS
51	Newey-West HAC OLS	rel $\leq 1.4 \times 10^{-15}$ (post HAC sandwich sqrt(n) scaling fix)	rel $\leq 8.5 \times 10^{-16}$	PASS
52	Classical SCM (unique solution)	rel $\leq 7.8 \times 10^{-8}$ (identified convex SCM; sp and Stata synth recover exact weights+gap)	rel $\leq 7.7 \times 10^{-8}$	PASS
53	Cluster-robust CR2 / CR3 SE	abs $\leq 1.84 \times 10^{-8}$ (CR2 and analytic CR3 match clubSandwich)	abs $\leq 5.93 \times 10^{-9}$	PASS
54	Two-way cluster-robust SE	abs $\leq 7.84 \times 10^{-16}$ (matches sandwich::vcovCL(HC1,cadjust); fixest min-G df convention differs $\sim 10^{-3}$)	abs $\leq 1.33 \times 10^{-15}$	PASS
55	HC2 / HC3 robust SE	abs $\leq 6.39 \times 10^{-9}$	abs $\leq 4.01 \times 10^{-13}$	PASS
56	Three-way cluster-robust SE	abs $\leq 2.09 \times 10^{-15}$	abs $\leq 7.77 \times 10^{-16}$	PASS
57	Binary logit ML	rel $\leq 2.7 \times 10^{-11}$	rel $\leq 2.7 \times 10^{-11}$	PASS
58	Poisson ML	rel $\leq 9.2 \times 10^{-15}$	rel $\leq 8.7 \times 10^{-12}$	PASS
59	LIML k-class IV	rel $\leq 1.7 \times 10^{-15}$ (ivregress runs with ‘small’ (RSS/(n-k)))	rel $\leq 9.7 \times 10^{-16}$	PASS
60	SUR one-step FGLS	rel $\leq 1.5 \times 10^{-14}$ (Sigma divisor n (sureg default / noDfCor))	rel $\leq 1.5 \times 10^{-15}$	PASS
61	Beta regression ML	rel $\leq 2.2 \times 10^{-12}$ (betareg SEs are expected-information (documented))	rel $\leq 2.1 \times 10^{-8}$	PASS
62	Truncated regression ML	rel $\leq 3.5 \times 10^{-10}$	rel $\leq 7.9 \times 10^{-8}$	PASS
63	Zero-inflated Poisson	rel $\leq 1.2 \times 10^{-7}$	rel $\leq 2.8 \times 10^{-8}$	PASS

(continued on the next page)

(Table 17 continued)

ID	Method	Worst diff vs R	Worst diff vs Stata	Verdict
64	Zero-inflated NB	rel $\leq 9.5 \times 10^{-7}$ (flat ZINB likelihood near optimum (1×10^{-5} budget))	rel $\leq 4.5 \times 10^{-11}$	PASS
65	Spatial ML (SAR/SEM/SDM)	rel $\leq 8.3 \times 10^{-8}$ (vs spatial-reg::lagsarlm/errorsarlm/Durbin, row-std rook W)	rel $\leq 5.1 \times 10^{-8}$	PASS
66	Spatial GMM (SAR-2SLS/SEM-GMM)	rel $\leq 4.6 \times 10^{-8}$ (vs spatial-reg::stsls(W2X=F)/GMerrorsar; SEM point-only)	rel $\leq 7.3 \times 10^{-16}$	PASS
67	Panel GLM (feglm / fepois) [†]	rel $\leq 9.7 \times 10^{-9}$ (vs fixest::feglm/fepois, absorbed id FE; IWLS SE 1×10^{-5})	rel $\leq 1.8 \times 10^{-9}$	PASS
68	Within transformation	rel $\leq 3.5 \times 10^{-15}$ (sp.demean(solver='map') vs textbook mean-within)	rel $\leq 8.8 \times 10^{-15}$	PASS
69	Panel balance filter	rel ≤ 0 (sp.balance_panel vs base R counts == n_periods)	rel ≤ 0	PASS
70	Policy tree (exact, depth 1–2)	rel $\leq 9.6 \times 10^{-16}$ (shared AIPW scores; exact welfare optimum vs policytree, all 1200 per-row policy decisions identical at both depths)	rel $\leq 1.4 \times 10^{-16}$	PASS
71	DML family (IRM / PLIV / IIVM)	rel $\leq 9.1 \times 10^{-11}$ (shared explicit fold_id across all three DoubleML model classes; PLIV at the floating-point floor, IRM/IIVM limited by the logistic-MLE optimiser)	rel $\leq 6.9 \times 10^{-7}$	PASS
72	TMLE (targeting step)	rel $\leq 1.9 \times 10^{-9}$ (shared initial Q / g1W; per-arm fluctuation matching tmle::tmle's submodel)	rel $\leq 1.9 \times 10^{-9}$	PASS
73	Gardner two-stage DiD	rel $\leq 4.8 \times 10^{-8}$ (estimate rel 4.8×10^{-8} and SE rel 2.7×10^{-10} against did2s (Stata did2s: 2.4×10^{-12} / 1.3×10^{-14}); vce='analytic' is the did2s corrected two-stage clustered variance, so stage-1 estimation error is propagated on all three sides)	rel $\leq 2.4 \times 10^{-12}$	PASS
74	Changes-in-Changes (ATT + QTE)	rel $\leq 2.4 \times 10^{-15}$ (machine precision vs qte::CiC (worst 4.4×10^{-15}))	rel ≤ 0.006 (Stata convention gap)	PASS
75	Stacked DiD (never-treated controls)	rel $\leq 3.9 \times 10^{-13}$ (rel $< 1 \times 10^{-6}$ vs a hand-written fixest stack)	rel $\leq 7.1 \times 10^{-13}$	PASS
76	Pre-trends power (Roth 2022)	rel $\leq 4 \times 10^{-5}$ (rel $< 1 \times 10^{-4}$ vs pretrends::pretrends; the 1×10^{-3} budget covers mvtnorm::pmvnorm's randomised integrator)	rel ≤ 0.00014	PASS

(continued on the next page)

(Table 17 continued)

ID	Method	Worst diff vs R	Worst diff vs Stata	Verdict
77	Triple differences ATT(g,t)	$\text{rel} \leq 5.7 \times 10^{-15}$ (machine precision vs triplediff::ddd (worst 1×10^{-12}), estimates and analytic SEs, across dr / ipw / reg)	<i>no Stata implementation: the Ortiz-Villavicencio/Sant’Anna DDD estimator ships as the R package triplediff. Re-checked 2026-08-06: neither ‘tripleddiff’ nor ‘ddd’ resolves locally and ‘ssc describe tripleddiff’ returns r(601).</i>	PASS
78	dCDH intertemporal event study	$\text{rel} \leq 3.3 \times 10^{-15}$ (machine precision vs DIDmultiplegtDYN (worst 5×10^{-15}), across absorbing and switch-off designs)	$\text{rel} \leq 2.1 \times 10^{-15}$	PASS
79	Functional-form test (Roth-Sant’Anna)	$\text{rel} \leq 1.3 \times 10^{-14}$ (machine precision vs didFF (worst 1.2×10^{-15}); the module’s 1×10^{-3} budget covers only the simulated p-value)	<i>no Stata implementation: the Roth-Sant’Anna functional-form test ships as the R package didFF (GitHub only, not CRAN). Re-checked 2026-08-06: ‘ssc describe didff’ returns r(601), not found.</i>	PASS
80	Continuous-treatment DiD (CGS)	$\text{rel} \leq 2.4 \times 10^{-14}$ (machine precision vs contdid::cont_did (worst 1×10^{-12}))	<i>no Stata implementation: the CGS continuous-treatment estimator ships as the R package contdid (GitHub only, not CRAN). Re-checked 2026-08-06: ‘ssc describe contdid’ returns r(601), not found.</i>	PASS
81	dCDH 2020 DID_M (on/off switching)	$\text{rel} \leq 3.9 \times 10^{-15}$ (machine precision vs archived DIDmultiplegt 0.1.4)	$\text{rel} \leq 3.2 \times 10^{-15}$	PASS
82	Design-based staggered rollout (Roth-Sant’Anna)	$\text{rel} \leq 9.1 \times 10^{-16}$ (machine precision on all 33 rows (worst 3.7×10^{-15}) against both staggered 1.2.2 in R and the SSC staggered port in Stata, including both the Neyman and adjusted standard errors)	$\text{rel} \leq 5.7 \times 10^{-16}$	PASS
83	LP-DiD event study (Dube-Girardi-Jorda-Taylor)	$\text{rel} \leq 5 \times 10^{-15}$ (estimate and SE both $\text{rel} < 1 \times 10^{-14}$ vs a direct transcription)	$\text{rel} \leq 2.5 \times 10^{-15}$	PASS
84	BJS pre-treatment lead vector (Borusyak-Jaravel-Spiess)	$\text{rel} \leq 5.1 \times 10^{-8}$ (point estimates $\text{rel} < 1 \times 10^{-7}$ against both references; the three leads match Stata <code>pretrends(3)</code> at $\text{rel} < 1 \times 10^{-14}$ on estimate and SE, and are a py↔Stata pin because R exposes no equivalent option; horizon SEs are an open gap)	$\text{rel} \leq 5.1 \times 10^{-10}$	PASS
85	Dynamic TWFE event study (benchmark specification)	$\text{rel} \leq 3.2 \times 10^{-13}$ (estimates three-way to 5.7×10^{-14} ; SE matches <code>fixest</code> to 9.3×10^{-14} under <code>ssc(fixef.K="none")</code> , and <code>reghdfe</code> ’s SE is reproduced from it by the exact $(N - K_{py})/(N - K_{St})$ d.f. scalar)	$\text{rel} \leq 1.5 \times 10^{-14}$	PASS

(continued on the next page)

(Table 17 continued)

ID	Method	Worst diff vs R	Worst diff vs Stata	Verdict
86	fect counterfactual estimators (fe / ife / mc)	rel $\leq 1.8 \times 10^{-13}$ (native port of fect's EM map; three outcome models on one staggered panel)	rel $\leq 9.8 \times 10^{-10}$	PASS
87	interflex marginal effects (linear / binning / kernel)	rel $\leq 4 \times 10^{-15}$ (native port incl. R's density() grid for the adaptive kernel bandwidth)	rel $\leq 1.5 \times 10^{-14}$	PASS
88	RD bandwidth selection (all ten CCT selectors)	rel $\leq 9.4 \times 10^{-13}$ (all 68 bandwidths agree with rdrobust to 1.8×10^{-12} and with Stata to 3.7×10^{-9} ; certwo is pinned against R only because Stata rdbwselect 10.0.0 exits r(3200) on that selector, including on the package authors' own rdrobust_senate.dta)	rel $\leq 3.5 \times 10^{-9}$	PASS
89	Multi-score / geographic RD at boundary points	rel $\leq 1.2 \times 10^{-12}$ (three boundary points along one boundary; bias-corrected and conventional estimates, standard errors, selected bandwidths and per-side effective sample sizes agree with rdmulti to 7.6×10^{-12} and with Stata to 3.3×10^{-9} , and the six effective sample sizes are exactly equal on all three sides)	rel $\leq 5.4 \times 10^{-10}$	PASS

Table 17: Every Track A module, at **StatsPAI** 1.32.0. The ID column is the module's two-digit prefix in **tests/r_parity/**. The two difference columns report the worst residual gap across that module's headline rows, point estimates only; per-row standard errors, confidence bounds, and documented convention gaps are in the generated **tests/r_parity/results/parity_table_3way.md**. Italic text in the **Stata** column is the measured reason no portable **Stata** artifact exists for that module, not an untested assumption of unavailability. All 89 modules are joined to a canonical R reference and 85 additionally to a canonical or audited **Stata** bridge, every side reading the same CSV bytes.

Of the 89 modules, all but one receive a pass-type verdict. A pass-type verdict covers ordinary strict passes, common-specification passes such as RD at a forced common bandwidth, seeded stochastic passes such as DML, and coefficient-level passes whose secondary diagnostic follows a documented small-sample convention. The single remaining *GAP* is the classical-SCM donor-weight non-uniqueness disclosure of Section 5.4, kept visible rather than absorbed into a wider tolerance. Two things this table does not do: show per-row detail, or show that either side is re-derivable (Section 8.2).

B. Output-changing corrections

Each fix in Table 18 is pinned by a regression test; users on affected versions should re-estimate. Section 5.9 discusses 1.23.0 and 1.25.0. The 1.29.0 count is the numbered items in **docs/dev/campaign_phase3/**, each counted once, excluding documentation-only corrections.

Version	Affected area	Output-changing fix
1.6–1.15	IV/LIML/Heckman, event study, CS-DiD, TMLE, DML, meta-learners	Small-sample, two-step and rank SE corrections; influence-function, orthogonal-score and AIPW fixes.
1.16.0	Quantile reg., Newey–West, Arellano–Bond, forest ATE	SE scaling and GMM weight construction corrected; forest headline moved to the grf AIPW estimand.
1.21.0	CBPS, entropy balancing, Mahalanobis, genetic matching	GMM basis, exact moment balance, and covariance conventions corrected (up to $8.6\times$ off CBPS); now pinned against CBPS , ebal , MatchIt , optmatch , Matching , sbw .
1.23.0	CS-DiD unit weights; IPW SEs	Weighted estimand restored; IPW SEs were 9–11% (up to 89% weighted) too large; pinned to did::att_gt (see text).
1.24.0	Sun–Abraham, GLMM, PPML-HDFE, TWFE SEs	Degrees-of-freedom/nesting conventions fixed to the reference rules; SEs moved 0.1–2% (see text).
1.25.0	Forest average, continuous treatment	The binary AIPW score was applied to a continuous treatment (-1266.6 , $p = 0.0000$ on Card); guarded in 1.25.0, replaced by the Riesz-representer average in 1.29.0. Binary results unchanged.
1.26.0	Evidence grades	A false alias certification (sp.wooldridge_did as sp.etwfe) withdrawn; grades, not estimates, changed.
1.27.0	ETWFE, etregress , RE logit/probit, FGLS, spatial LM	Cohort ATIs read from a saturated regression; clustered SEs account for absorbed effects; the etregress default is a true MLE with first-stage-aware two-step SEs; lm_tests no longer reverses its model-selection conclusion.
1.28.0	SCM placebo p -values; MR; evidence grades	Permutation p -values now rank the treated unit with its placebos, $(1 + \#\{\text{placebo} \geq \text{treated}\})/(J + 1)$; MR estimators ported to their reference definitions; 31 functions graded T2 on closed-form tests that never ran R or Stata were regraded T1 (see text).
1.29.0	Parity campaign; Track A closure	Fourteen families bisected against their reference on identical bytes; 174 output-changing defects fixed, none caught by a unit test; forest defaults follow grf ; Gardner two-stage SE as did2s ; PSM SE as teffects psmatch .
1.30.0	HonestDiD on a CS fit; forest ATT/ATC	The event-study covariance passed to sp.honest_did dropped the cohort-share term off the diagonal (cross-cell entries up to 8% off did), overstating robustness; now matches did to 2.8×10^{-13} , and the bounds of Listing 6 moved in the fourth decimal. ATT and ATC now raise for a continuous treatment, as in grf .
1.31.0	Entropy balancing; IV and HDFE covariances; CS event study; pre-trend tests; forest policy; audit; grades	sp.ebalance SE was twice the Monte Carlo SD; now the M-estimation sandwich (WeightIt to 3×10^{-9}). sp.iv(vce=) silently returned CR1 for CR2/CR3/wild requests; now bit-identical to sp.ivreg or refused. sp.fast.feols defaults to fixest ’s small-sample factor (the old default over-covered, 0.979). The convenience event study of a Callaway–Sant’Anna fit omitted the cohort-share term of its SEs (up to 0.9% on mpdta , 47% on a castle-doctrine panel); it now equals sp.aggte to 10^{-15} , and the grid of Listing 6 moved in the fourth decimal. sp.pretrends_test on an event study used a diagonal pre-period covariance ($p = 0.603$ against Stata’s 0.293); it now uses the joint one. sp.rate_split and sp.forest_policy_tree aggregate 21 sample splits instead of reporting one. Over-identification audit skipped on just-identified fits; native relative-magnitudes Honest-DiD; two parity rows that ran R itself rebuilt natively.

1.32.0	Silently ignored options; IV weights; quantile, tobit and panel SEs; margins; labels; original-data ledger	Options accepted and then dropped now take effect or raise: <code>sp.iv/sp.ivreg</code> returned unweighted estimates for <code>weights=</code> (4.238 weighted against 3.547 reported; now <code>Stata ivregress [aw=]</code> to 10^{-14}); <code>alpha=</code> on 31 estimators still printed 95% intervals; <code>sp.panel</code> ignored <code>weights=</code> and clustered a named <code>cluster=</code> column on the entity; <code>sp.synth</code> dropped <code>covariates=</code> for matrix completion and <code>standardize=False</code> for the classical method. A fuzzy <code>sp.rdrobust</code> with a <code>comb</code> selector chose the sharp bandwidth (now R to 10^{-12}). The <code>sp.qreg</code> default SE matched neither <code>Stata</code> nor <code>quantreg</code> (3–7% off on Track A); it is now <code>Stata's vce(iid)</code> , with $t(N - k)$ inference, and module 40's SE budget tightens from 10^{-1} to 10^{-6} . <code>sp.tobit</code> stopped short of the optimum (module 41: 2×10^{-6} to 3×10^{-11}). <code>sp.feols</code> intervals use <code>fixest's</code> degrees of freedom, CRE random-effects variance components follow <code>Stata</code> , and <code>sp.margins</code> averages over the estimation sample with the fit's weights. The DML PLR estimate is labelled <code>theta</code> , not ATE (a variance-weighted average, or a partial coefficient for continuous D), and <code>sp.cate_summary</code> reports <code>Mean</code> , not <code>Mean (ATE)</code> ; estimates unchanged. The original-data RD row ran the official <code>rdrobust</code> port and so compared the authors' code with itself; it now runs the native selector, and the call trace covers that ledger.
--------	---	--

Table 18: Output-changing correctness fixes through release 1.32.0; `CHANGELOG` and `MIGRATION.md` hold the detail.

Affiliation:

Biaoyue Wang (corresponding author)

Rural Education Action Program, Stanford Center on China's Economy and Institutions,
Stanford University, United States
and StatsPAI Inc., United States

E-mail: brycew6m@gmail.com, brycew6m@stanford.edu

URL: <https://github.com/brycewang-stanford/StatsPAI>

Scott Rozelle

Rural Education Action Program, Stanford Center on China's Economy and Institutions,
Stanford University, United States
and StatsPAI Inc., United States

E-mail: rozelle@stanford.edu